Next Gen Graphics

для SnowRunner (до октября 2026 года SnowRunner Next Gen).
Руководство: для новичков и подробно о каждой карточке

Для Next Gen 1.0.2 и всех изменений после него. Тексты карточек взяты из самой программы, из сборки от 2026-11-06 10:07.
Карточек: 34, строк настроек: 31.

Содержание

1. **Часть 1. Next Gen для чайников**
    1. Что такое Next Gen
    2. Что понадобится
    3. Установка шаг за шагом
    4. Что включить
    5. Передумали или хотите удалить всё
    6. После обновления игры
    7. Другие моды
    8. Если что-то пошло не так
2. **Часть 2. Окно программы**
3. **Часть 3. Все карточки вкратце**
    1. Light and shadow
    2. Reflections and water
    3. Atmosphere and image
    4. Scenery
    5. Interface
4. **Часть 4. Как это устроено, подробно**
    1. Как Next Gen попадает в игру
    2. Солнечные тени: разрешение, края, мягкость, швы
    3. Контактные тени
    4. Тени от фар
    5. Фоновое затенение: GTAO и широкий радиус
    6. Переотражённый свет
    7. Фоновое освещение неба и заполняющий свет
    8. Object reflections: проход отражений, reflection pass (SSSR)
    9. Вода: отражения, эффект Френеля, цвет глубины, свечение гребней
    10. Отражения на мокрой земле и ограничение бликов фар
    11. Headlights in reflections
    12. Объёмный туман и солнечные лучи
    13. Тонмаппинг, блум и цветокоррекция
    14. Кубические карты отражений из фотографий
    15. Ночное небо, яркие звёзды и небо
    16. Дым: свечение, форма, мягкие края
    17. Более чёткий дым и частицы
    18. Погода, мусор и брызги
    19. Детализация окружения и дальность травы
    20. Дни карт (Map days)
    21. Список загрузки: как пак сообщает игре, что в нём лежит
    22. Логотип Next Gen и запуск без заставки
    23. Движок: сборка, проверка, откат
    24. Известные проблемы и исправления
5. **Часть 5. Что меняется на вашем ПК**
6. **Часть 6. Лицензия и авторы**

Часть 1. Next Gen для чайников

Эта часть для всех: что делает Next Gen, как его установить и что включить. В части 3 у каждой карточки есть своя страница, написанная так же просто; часть 4 объясняет, как устроена каждая из них, с формулами.

Что такое Next Gen

Next Gen это бесплатный графический мод для SnowRunner. Небольшая программа, **SnowRunnerNextGen.exe**, меняет несколько файлов самой игры на вашем ПК: шейдеры, которые рисуют свет, тени, воду и дым, а также часть игровых текстур и настроек. После этого игра рисует более чёткие тени, затенение там, где предметы соприкасаются, отражения на краске и воде, туман, который светится в сторону солнца, дым с освещённой стороной и более правдоподобное ночное небо.

Кроме одной небольшой DLL, SnowRunner Shadows, рядом с игрой ничего не запускается. ReShade и другие загрузчики не нужны, а программа сохраняет копию каждого файла, который меняет, так что всё всегда можно вернуть.

Что нужно

- SnowRunner из Steam, Epic Games или Microsoft Store, на Windows.
- Место на системном диске для копий оригинальных файлов: до 4 ГБ, смотря что вы включите. Один shared.pak весит 2 ГБ, и меняет его только карточка Scenery detail.
- Закрытая игра, пока вы устанавливаете, меняете или удаляете что-либо. Пока игра запущена, программа ничего не записывает. Если игру запустить во время Apply, установка остановится: закройте игру и снова нажмите Apply, уже сделанное останется на месте.

Установка по шагам

1. Скачайте свежий zip и архив с небом, **SnowRunnerNextGen_Skies.zip**, со страницы релизов проекта, **github.com/andizla/snowrunner-next-gen/releases**.
2. Закройте SnowRunner.
3. Распакуйте папку куда угодно, например на рабочий стол, распакуйте архив с небом в ту же папку и запустите **SnowRunnerNextGen.exe**. Больше ничего устанавливать не нужно.
4. В строке вверху окна показана папка игры, которую нашла программа. Если она ничего не нашла или нашла не ту, нажмите **Change** и выберите папку, в которой лежит SnowRunner.
5. Нажмите **Defaults**, затем **Apply**. Первый Apply читает собственные шейдеры игры и занимает минуту-другую. Потом появится окно с надписью Installed.
6. Запустите игру.

Чтобы обновиться, распакуйте новую версию и её архив с небом в отдельную папку и нажмите **Apply**: установка, сделанная старой версией, обновится на месте. Окно показывает то, что установлено, поэтому карточка, появившаяся в этой версии, будет без отметки; **Defaults** отмечает все карточки так, как они идут по умолчанию.

Что включить

Defaults даёт рекомендуемый набор: карточки, отмеченные «вкл.» в таблице ниже (всего 31). Начните с него и поиграйте какое-то время.

Карточки, выключенные по умолчанию (их 3), выключены не просто так: они экспериментальные, на любителя или сильнее снижают частоту кадров. У каждой есть своя страница в части 3: что она делает, что вы заметите и чего она стоит. Включите одну, нажмите Apply и посмотрите в игре. С установленной SnowRunner Shadows клавиша **F8** выключает и включает все шейдерные эффекты, так что можно сравнить с одного и того же места.

У некоторых карточек есть строки с кнопками: разрешение, сила эффекта или карты, к которым относится изменение. Значения, с которыми карточка стоит изначально, и есть рекомендуемые. В части 3 перечислены все строки с их значениями.

Карточка	По умолчанию	Что делает
Light and shadow		
SnowRunner Shadows	**вкл.**	Передаёт отражениям и переотражённому свету нужное им изображение сцены и может рисовать солнечную текстуру теней в большем разрешении, чтобы тени были детальнее: 1x оставляет игровой размер.
Shadow edges	**вкл.**	Заново построенные прямые края теней, без лесенки из текселей, при любом разрешении.
Blocker search	**вкл.**	Тени резкие в месте контакта и тем мягче, чем дальше они падают от предмета, который их отбрасывает.
Contact shadows	**вкл.**	Мелкие солнечные тени, которые трассируются по экрану там, где предметы соприкасаются или перекрываются: для таких деталей текстура теней слишком грубая.
Headlight shadows	**вкл.**	Ваши фары отбрасывают тени: столбы, деревья и камни перекрывают луч, а не дают ему осветить то, что стоит за ними.
GTAO	**вкл.**	Фоновое затенение GTAO (ground-truth ambient occlusion) вместо игрового SSAO: мягкие контактные тени в углах, под грузовиками и в колёсных арках.
Wide occlusion	**вкл.**	Поиск GTAO продолжается до 6 м, но затеняет мягко: лёгкая тень под грузовиками, крышами и кронами деревьев, а контактные тени не меняются.
Sky ambient	**вкл.**	Заполняющий свет берёт цвет неба вместо ровного серого.
Fill light	**вкл.**	Меньше ровного света, заполняющего тень: затенённые стороны, кроны деревьев изнутри и грузовики в тени становятся темнее, а освещённые солнцем места и небо остаются как есть.
Bounce light	выкл.	Освещённые солнцем земля и краска подсвечивают своим цветом то, что стоит рядом: под грузовиками, в колёсных арках, у подножия стен. Экспериментальная карточка: местами свет получается слишком сильным.
Reflections and water		
Object reflections	**вкл.**	Краска, стёкла и хром отражают окружающую сцену, а не нарисованное небо.
Headlights in reflections	**вкл.**	Ночью краска, которой на экране нечего отразить, показывает тёплый свет фар.

Карточка	По умолчанию	Что делает
Water reflections	**вкл.**	В реках, грязи, озёрах, морях и лужах отражаются берега, деревья и грузовики, а не только небо; под пологим углом цвет воды уступает место отражению.
Wet ground reflections	**вкл.**	Мокрая грязь и глянцевая земля отражают окружающую сцену: чётко на блестящих участках, мягко на шероховатой грязи.
River depth colour	**вкл.**	Глубокая вода окрашивается своим цветом в бирюзовый или бурый, а не выцветает в серый.
Sun glow through waves	**вкл.**	Речные волны между вами и низким солнцем светятся цветом самой воды там, где солнце действительно до них достаёт.
Headlight glare cap	выкл.	Блеск фар на мокрой земле и в лужах спокойнее, без длинной яркой полосы поперёк.
Reflection cubes	**вкл.**	Нарисованные небеса, которые отражают краска, стёкла и хром и из которых Sky ambient и Bounce light берут цвет неба и земли, сделаны заново по фотографиям: 25 кубмап из 21 панорамы, для ясных и непогожих дней, сумерек, рассвета и ночи каждого региона. Каждая сохраняет яркость игровой кубмапы, а в снежных регионах на земле снег.
Atmosphere and image		
Volumetric fog	выкл.	Без полос и со свечением в сторону солнца. Можно включить Sun shafts, солнечные лучи сквозь деревья и пыль: они экспериментальные и видны также поверх вашего грузовика и других предметов рядом с камерой.
Weather	**вкл.**	Больше собственной погоды игры: движущиеся тени облаков на всех картах, дожди, которые то усиливаются, то стихают, морось вечером и ночью, облака на горизонте, дождь, видимый дальше (по умолчанию выключен), светлячки ночью и пыльца днём.
Map days	**вкл.**	В четырёх регионах нет своих дней, и они показывают дни другого региона. Мэн показывает дни двух снежных регионов: снег в воздухе и зимний свет на карте без снега. Карточка даёт каждой их карте подходящие дни, подобранные для каждой карты отдельно (map by map), со строкой для каждого региона.
Smoke glow	**вкл.**	Выхлоп и другой дым светятся, когда солнце позади них.
Smoke shape and softer edges	**вкл.**	У клубов дыма и пыли появляются освещённая и затенённая стороны, а спрайты мягче растворяются там, где касаются земли.
Sharper smoke and particles	**вкл.**	Игровые текстуры дыма, пыли, брызг и искр в удвоенном размере, чтобы частицы оставались чёткими вблизи.
Debris and splashes	**вкл.**	Листья, трава, ветки и грязь из-под колёс полминуты лежат на земле, и шины их толкают, а не исчезают через четыре секунды. Брызги грязи, воды и снега дают больше капель, которые дольше лежат и оставляют больше следов на грузовике. Снежная пыль висит в воздухе. Выхлоп и пыль тоже держатся дольше.
Night sky	**вкл.**	Звёздная карта NASA вместо игровой текстуры звёзд: звёзды настоящего неба тонкими точками и свечение Млечного Пути.

Карточка	По умолчанию	Что делает
Skies	**вкл.**	Нарисованные небеса всех регионов сделаны заново по фотографиям настоящего неба: 49 игровых небесных куполов, полос облаков и ночных слоёв, каждый со своим размером и яркостью; солнце с фотографий убрано, так что единственное солнце здесь игровое. Северное сияние зелёное, как настоящее. Облака по-прежнему плывут.
Brighter stars	**вкл.**	Звёзды ночного неба ярче и лучше видны на тёмном фоне.
Tonemap	**вкл.**	Сильнее контраст и слабее блум.
Bloom soft cut	**вкл.**	Никакой дымки от яркого снега и неба.
Photo grade	**вкл.**	Цвет ближе к фотографии: зелень, осенняя листва и фиолетовая дымка спокойнее, белое чуть темнее, нет грязи на объективе поверх неба. Встроено в собственную цветокоррекцию игры.
Scenery		
Scenery detail	**вкл.**	Объекты окружения дольше сохраняют полную детализацию, так что на ходу меньше что-то меняется на глазах. All meshes делает shared.pak больше 2 ГБ. Shown far away: при plants растения (деревья, кусты, камни, ветки), при everything вообще всё больше не исчезает на заданном расстоянии. Small shadows: то, что не отбрасывает тень или теряет её в нескольких метрах, получает тень.
Grass reach	**вкл.**	Трава видна дальше. Снижает частоту кадров там, где много травы.
Interface		
Next Gen logo	**вкл.**	NEXT GEN под логотипом игры на титульном экране, первом экране загрузки, в главном меню, меню паузы и на карте.

Передумали или хотите удалить всё

Отметьте или снимите отметки с карточек и снова нажмите **Apply**: программа собирает файлы ровно под то, что отмечено, каждый раз из оригинальных файлов игры. **Restore originals** возвращает каждый файл в том виде, в каком он был до Next Gen, и удаляет SnowRunner Shadows. Копии оригинальных файлов остаются в **%LOCALAPPDATA%\SnowRunnerNextGen** (от 2 до 4 ГБ): чтобы удалить Next Gen совсем, после Restore удалите эту папку вручную.

После обновления игры

Обновление игры или проверка файлов игры в магазине возвращает собственные файлы игры, и изменения Next Gen пропадают вместе с ними. Запустите SnowRunnerNextGen.exe и снова нажмите **Apply**.

Одной карточке это нужно раньше других: если в карточке Map days для Maine стоит значение «map by map», после такого обновления Мэн перестаёт загружаться, пока вы снова не нажмёте Apply. На странице карточки сказано почему.

Другие моды

Если другой мод, например текстурпак, изменил один из файлов, с которыми работает Next Gen, программа спросит, прежде чем писать. **Take it as the original**: Next Gen запишет свои изменения поверх файла в его нынешнем виде, а Restore потом вернёт это модифицированное состояние. **Leave it out**: этот файл и его карточки останутся как есть, а остальное будет применено.

Если что-то пошло не так

Каждый Apply и Restore записывается в **install.log** в папке **%LOCALAPPDATA%\SnowRunnerNextGen**: что программа нашла и каждый её шаг. В окне с результатом кнопка **Show the log** открывает эту папку. Приложите этот файл, когда сообщаете о проблеме.

Поддержка, вопросы и разговоры о моде: канал на Discord-сервере EnthusiastRunner, спасибо EnthusiastRunner за него.

Часть 2. Окно программы

Next Gen Graphics устанавливает, меняет и удаляет графические модули этого проекта, всё из одного окна.

Выбор

Каждая карточка это один модуль. Если отметить модуль, которому нужен другой, отметится и тот, а если снять отметку с модуля, она снимется и с тех, кому он нужен. Кнопка (?) на карточке рассказывает, что она делает. Defaults выбирает рекомендуемый набор.

Apply

Собирает файлы игры ровно под то, что отмечено, из оригинальных файлов игры и устанавливает их. Чтобы убрать модуль, снимите с него отметку и снова нажмите Apply.

Restore originals

Возвращает файлы в то состояние, в каком они были до изменений Next Gen Graphics, и удаляет SnowRunner Shadows.

Резервные копии

Перед первым изменением shader.pak, shared.pak, initial.pak, boot.pak и gfx.pak сохраняются как есть в %LOCALAPPDATA%\SnowRunnerNextGen, и каждая сборка начинается с этих копий. Рядом с файлами игры программа оставляет небольшую заметку, SnowRunnerNextGen.json, где названы файлы с её изменениями. Если сохранённые копии когда-нибудь пропадут (например, после переустановки Windows), по заметке программа поймёт, что эти файлы не оригинальные: тогда она ничего не меняет и просит сначала проверить файлы игры через магазин.

Другие моды

Если другой мод (например, текстурпак) или обновление игры с тех пор изменили один из этих файлов, Apply и Restore сначала спрашивают. Можно принять файл в нынешнем виде за оригинал (Take it as the original), и Next Gen Graphics запишет свои изменения поверх него; тогда Restore вернёт его в это модифицированное состояние, вместе с изменениями другого мода, а не в оригинальный файл игры. Или можно его пропустить (Leave it out): файл и его модули останутся как есть.

Перед началом

Закройте игру: пока она запущена, её файлы нельзя изменить.

Если что-то пошло не так

Каждый запуск Apply и Restore записывается в install.log в %LOCALAPPDATA%\SnowRunnerNextGen: что программа нашла и какой шаг сделала. После остановки или замечания кнопка Show the log в окне результата открывает эту папку. Приложите этот файл, когда сообщаете о проблеме.

Сравнение в игре

Когда установлен SnowRunner Shadows: F8 выключает и включает все шейдерные эффекты, F4 контактные тени, F5 переключает проход фонового затенения между половинным и полным размером, F6 включает и выключает проход отражений, F9 отражения на мокрой земле, F10 переотражённый свет, а F11 переключает между GTAO и игровым SSAO. Первое нажатие F8 или F11 занимает какое-то время: в этот момент создаются собственные шейдеры игры. Пока зажаты Alt, Ctrl

или Shift, клавиши ничего не делают. Если установлен Next Gen Gameplay или Next Gen Tire Inflation, те же переключатели находятся на вкладке Graphics рядом с их вкладками в меню NEXT GEN (F2), а эти клавиши молчат, потому что часть из них заняты другими модами; KeysWithMenu=1 в SnowRunnerShadows.ini оставляет их рабочими. Если установлен только Next Gen Graphics, меню нет, и клавиши работают как раньше.

Часть 3. Все карточки вкратце

Light and shadow

SnowRunner Shadows

Shadows · включена по умолчанию · подробно: Как Next Gen попадает в игру, Солнечные тени: разрешение, края, мягкость, швы

Передаёт отражениям и переотражённому свету нужное им изображение сцены и может рисовать солнечную текстуру теней в большем разрешении, чтобы тени были детальнее: 1x оставляет игровой размер.

Строка	Значения (по умолчанию выделено жирным)
Resolution	**1x** · 2x · 3x · 3.5x
Self-shadow fix	**on** · off

ЧТО ДЕЛАЕТ

Солнечные тени перед каждым кадром рисуются в одну текстуру теней. Это дополнение передаёт изображение сцены переотражённому свету и отражениям, поэтому они без него не работают, и может увеличить текстуру теней, до 3.5 раза по каждой стороне, чтобы каждая тень рисовалась большим числом пикселей. Это один небольшой файл рядом с игрой (hid.dll), и он не меняет ни одного файла самой игры.

ЧТО ВЫ ЗАМЕТИТЕ

На 2x и выше: детальнее тени деревьев, заборов, кранов и грузовиков, в том числе вдали.

ЧЕГО СТОИТ

На 1x текстура теней ничего не стоит. Увеличенная стоит времени GPU, пока рисуются тени, и видеопамяти (несколько сотен МБ на 3.5x). Замеры в 4K на RTX 4080 со всеми остальными модулями: 2x 43 FPS, 3x 37 FPS, 3.5x 33 FPS против 49 FPS в игровом размере.

НАСТРОЙКИ

Resolution: 1x оставляет игровую текстуру теней, а Shadow edges сохраняет её края прямыми; 2x, 3x и 3.5x рисуют её крупнее ради детализации, с частотой кадров, указанной выше. Self-shadow fix: подстраивает игровое смещение теней (bias) под увеличенную текстуру, чтобы крутые скалы не затеняли сами себя пятнами. Оставьте включённым.

ПОЛЕЗНО ЗНАТЬ

В игре F8 выключает и снова включает все шейдерные эффекты этого установщика для быстрого сравнения. Увеличенная текстура теней остаётся.

Shadow edges

Shadows · включена по умолчанию · подробно: Солнечные тени: разрешение, края, мягкость, швы

Заново построенные прямые края теней, без лесенки из текселей, при любом разрешении.

Строка	Значения (по умолчанию выделено жирным)
Method	**rebuilt edges** · 16-tap filter
Cascade seams	**blended** · off

ЧТО ДЕЛАЕТ

В солнечной текстуре теней ограниченное число текселей, поэтому игра рисует край тени лесенкой из них. Rebuilt edges проходит вдоль каждого края по текселям и рисует прямую линию, которую они образуют, так что края получаются прямыми и чистыми при любом разрешении, даже при игровом. 16-tap filter, прежний метод, вместо этого сглаживает края 16 выборками на равномерной сетке.

ЧТО ВЫ ЗАМЕТИТЕ

Прямые чистые края у теней зданий, грузовиков, заборов и кранов, без лесенки.

ЧЕГО СТОИТ

Меньше, чем 16-tap filter: примерно на 0.7 мс на кадр быстрее при равных условиях, замер в 4K на RTX 4080. Rebuilt edges на 1x давали 64.8 FPS, тогда как 16-tap filter на 2x давал 59.7.

НАСТРОЙКИ

Method: rebuilt edges (по умолчанию) или 16-tap filter, одно из двух. Cascade seams: игра рисует солнечные тени ступенями детализации (каскадами), которые сменяются на заданных расстояниях, и там, где тени становятся грубее, может быть видна линия. Значение blended дизерингом смешивает последние несколько процентов каждой ступени со следующей, и линии больше не видно. Работает только с rebuilt edges.

ПОЛЕЗНО ЗНАТЬ

Круглые тени могут выходить слегка угловатыми, а у очень мягких концов теней остаётся немного лесенки.

АВТОРЫ

Метод rebuilt edges основан на работах revectorization-based shadow mapping (Macedo и Apolinário, 2016) и shadow map silhouette revectorization (Bondarev, 2014).

Blocker search

Shadows · включена по умолчанию · подробно: Солнечные тени: разрешение, края, мягкость, швы

Тени резкие в месте контакта и тем мягче, чем дальше они падают от предмета, который их отбрасывает.

ЧТО ДЕЛАЕТ

Настоящие солнечные тени резкие там, где предмет касается земли, и тем мягче, чем дальше от него падают. Карточка выясняет, насколько каждая затенённая точка удалена от того, что заслоняет солнце, и смягчает край соответственно.

ЧТО ВЫ ЗАМЕТИТЕ

Чёткие контактные тени под шинами и столбами, длинные тени деревьев с мягкими концами.

ЧЕГО СТОИТ

Немного: один поиск из 16 выборок там, где падают тени.

ПОЛЕЗНО ЗНАТЬ

Работает вместе с Shadow edges при любом методе.

Contact shadows

Shadows · включена по умолчанию · требует SnowRunner Shadows · подробно: Контактные тени

Мелкие солнечные тени, которые трассируются по экрану там, где предметы соприкасаются или перекрываются: для таких деталей текстура теней слишком грубая.

ЧТО ДЕЛАЕТ

Солнечная текстура теней слишком грубая для мелочей: дверная ручка, зеркало или камень не отбрасывают своей тени, а колесо будто парит чуть над землёй. Для каждого пикселя карточка ведёт короткий луч к солнцу через то, что есть на экране, и добавляет тень от всего, что этот луч перекрывает.

ЧТО ВЫ ЗАМЕТИТЕ

Колёса и грузовики стоят на земле, а мелкие детали вроде ручек, зеркал и болтов отбрасывают свои тени.

ЧЕГО СТОИТ

Немного времени GPU: ещё один проход по экрану.

ПОЛЕЗНО ЗНАТЬ

Требует SnowRunner Shadows, который выполняет этот проход. Отбрасывать такие тени может только то, что есть на экране. В игре F4 выключает и включает их.

АВТОРЫ

Изменённая копия screen-space shadows от Bend Studio (Copyright 2023 Sony Interactive Entertainment, Apache License 2.0).

Headlight shadows

Shadows · включена по умолчанию · требует SnowRunner Shadows · подробно: Тени от фар

Ваши фары отбрасывают тени: столбы, деревья и камни перекрывают луч, а не дают ему осветить то, что стоит за ними.

ЧТО ДЕЛАЕТ

Игра освещает всё в конусе ваших фар, даже то, что стоит за столбом или деревом. Для каждого освещённого пикселя карточка идёт от него к каждой из двух фар через то, что есть на экране, и ослабляет свет фары там, где что-то стоит на пути.

ЧТО ВЫ ЗАМЕТИТЕ

Ночью столбы, деревья, камни и другие грузовики отбрасывают длинные тени от ваших фар, а края теней наполовину освещены там, где светит только одна фара.

ЧЕГО СТОИТ

Немного времени GPU там, где светят фары: до 48 чтений глубины на каждый освещённый пиксель.

ПОЛЕЗНО ЗНАТЬ

Требует SnowRunner Shadows, который даёт глубину. Отбрасывать такие тени может только то, что есть на экране, и ничто ближе 2.5 м к фарам. Проверено в игре, настройка ещё продолжается.

GTAO

Light · включена по умолчанию · подробно: Фоновое затенение: GTAO и широкий радиус

Фоновое затенение GTAO (ground-truth ambient occlusion) вместо игрового SSAO: мягкие контактные тени в углах, под грузовиками и в колёсных арках.

Строка	Значения (по умолчанию выделено жирным)
Quality	half size · **full size**

ЧТО ДЕЛАЕТ

Фоновое затенение (ambient occlusion) это мягкая тень там, где предметы сходятся: под грузовиком, между шинами, в углах. Собственная версия игры это старый SSAO только по глубине, радиус которого на любом расстоянии занимает одну и ту же долю экрана: он рисует тёмный контур вдоль каждого края и серую плёнку на ровной земле. GTAO (ground truth ambient occlusion) смотрит в нескольких направлениях вокруг каждого пикселя, находит, какую часть неба оставляет открытой геометрия рядом, и затемняет ровно на эту величину, в пределах 1.5 м в мире игры.

ЧТО ВЫ ЗАМЕТИТЕ

Никаких тёмных контуров вдоль решёток, протекторов и перил, чистая открытая земля и мягкая тень там, где предметы действительно сходятся: между сдвоенными шинами, внутри рамы, где колесо касается земли. Вокруг тонких предметов вроде столбов, кабелей и перил больше нет ореола.

ЧЕГО СТОИТ

В режиме full size больше времени GPU, чем у игрового прохода. При half size SnowRunner Shadows рисует проход в половинном разрешении, затем размывает и масштабирует его: в 4K на RTX 4080 частота кадров выросла с 63.7 до 75.0 FPS.

НАСТРОЙКИ

Quality: full size (по умолчанию, самый чёткий) или half size (намного быстрее). Для half size нужен SnowRunner Shadows, который рисует проход; без него проход идёт в режиме full size.

ПОЛЕЗНО ЗНАТЬ

Фоновое затенение должно быть включено в настройках видео игры. На нём построен Bounce light. В игре F11 переключает между GTAO и игровым SSAO, а F5 между половинным и полным размером.

АВТОРЫ

Ground truth ambient occlusion: Jorge Jimenez, Xian-Chun Wu, Angelo Pesce и Adrian Jarabo (2016). Интегрирование по горизонту сделано по XeGTAO от Intel, авторы Filip Strugar и Steve Mccalla (MIT).

Wide occlusion

Light · включена по умолчанию · требует GTAO · подробно: Фоновое затенение: GTAO и широкий радиус

Поиск GTAO продолжается до 6 м, но затеняет мягко: лёгкая тень под грузовиками, крышами и кронами деревьев, а контактные тени не меняются.

ЧТО ДЕЛАЕТ

GTAO ищет в нескольких направлениях вокруг каждой точки то, что закрывает небо в пределах 1.5 м. Карточка продолжает тот же поиск до 6 м, а найденное там затемняет лишь слегка: вдвое слабее на 1.5 м и совсем не затемняет на 6. Так земля под грузовиком, стены под крышей и крона дерева изнутри получают мягкую тень, а контактные тени остаются ровно такими, какими их рисует GTAO.

ЧТО ВЫ ЗАМЕТИТЕ

Мягкое пятно тени под грузовиками и прицепами, тень под крышами и навесами, чуть больше глубины внутри деревьев.

ЧЕГО СТОИТ

Немного: около 1 мс на кадр в 4K (замер на RTX 4080). Каждый пиксель смотрит далеко в одном направлении и делится найденным с соседями.

ПОЛЕЗНО ЗНАТЬ

Требует GTAO. В игре F11 переключает между GTAO (вместе с этой карточкой) и игровым SSAO.

Sky ambient

Light · включена по умолчанию · подробно: Фоновое освещение неба и заполняющий свет

Заполняющий свет берёт цвет неба вместо ровного серого.

ЧТО ДЕЛАЕТ

Затенённые места освещаются одним заполняющим цветом на карту и время суток. Карточка даёт заполняющему свету цвет неба над головой: голубее под ясным небом, теплее к низкому солнцу. Яркость по-прежнему выбирает игра.

ЧТО ВЫ ЗАМЕТИТЕ

Тень в тон небу, а не ровно серая.

ЧЕГО СТОИТ

Совсем немного: несколько чтений изображения неба.

Fill light

Light · включена по умолчанию · подробно: Фоновое освещение неба и заполняющий свет

Меньше ровного света, заполняющего тень: затенённые стороны, кроны деревьев изнутри и грузовики в тени становятся темнее, а освещённые солнцем места и небо остаются как есть.

Строка	Значения (по умолчанию выделено жирным)
Strength	70 % · **55 %** · 85 %

ЧТО ДЕЛАЕТ

Тень в игре освещается сильным ровным заполняющим светом: на многих картах он почти такой же яркий, как солнце, поэтому затенённые стороны, кроны изнутри и грузовики в тени выглядят плоскими и блёклыми. Карточка приглушает этот заполняющий свет днём, в сумерках и на рассвете на каждой карте, чтобы тень получила глубину, как на фотографии. Ночь, гараж и меню сохраняют свой. Поскольку меньше заполняющего света затемняет всю картинку, адаптация глаза в игре подняла бы её обратно, и освещённые солнцем места стали бы ярче, чем раньше; поэтому вместе с ним снижается экспозиция того же освещения, и освещённые места остаются примерно такими, как были.

ЧТО ВЫ ЗАМЕТИТЕ

Глубже тень под деревьями и на теневых сторонах грузовиков и камней, больше глубины в листве, а освещённые места выделяются сильнее.

ЧЕГО СТОИТ

Ничего: карточка меняет только долю собственного заполняющего света игры.

НАСТРОЙКИ

Strength: какая доля заполняющего света остаётся. 55 %, значение по умолчанию, даёт глубокую тень, 70 % чуть слабее, 85 % лишь слегка.

ПОЛЕЗНО ЗНАТЬ

Меняет initial.pak вместе с Grass reach и сохраняет его изменения. Другие моды, которые меняют initial.pak, меняют тот же файл. Sky ambient по-прежнему окрашивает оставшийся заполняющий свет.

Bounce light

Light · выключена по умолчанию · требует GTAO, SnowRunner Shadows · подробно: Переотражённый свет

Освещённые солнцем земля и краска подсвечивают своим цветом то, что стоит рядом: под грузовиками, в колёсных арках, у подножия стен. Экспериментальная карточка: местами свет получается слишком сильным.

ЧТО ДЕЛАЕТ

Свет отражается. Освещённый солнцем снег подсвечивает днище грузовика, а красная краска подкрашивает землю рядом. Карточка добавляет это переотражение на малом расстоянии, примерно до 1.5 м, и собирает его каждый кадр из изображения сцены.

ЧТО ВЫ ЗАМЕТИТЕ

Меньше черноты под грузовиками и в колёсных арках в яркие дни и слабый цветной отсвет от краски и грязи.

ЧЕГО СТОИТ

Меньше 1 мс на кадр в 4K (замер на RTX 4080): свет собирается в проходе фонового затенения и добавляется ко всем освещённым материалам.

ПОЛЕЗНО ЗНАТЬ

По умолчанию выключена и экспериментальная: местами переотражение получается слишком сильным. На снегу и льду цвет берётся из кубической карты отражений, поэтому держите вместе с ней включённой Reflection cubes. Требует GTAO, в проходе которого работает, и SnowRunner Shadows, который передаёт изображение сцены. В игре F10 выключает и включает её.

Reflections and water

Object reflections

Reflections · включена по умолчанию · требует SnowRunner Shadows · подробно: Object reflections: проход отражений, reflection pass (SSSR)

Краска, стёкла и хром отражают окружающую сцену, а не нарисованное небо.

Строка	Значения (по умолчанию выделено жирным)
Method	**reflection pass** · march only

ЧТО ДЕЛАЕТ

Краска, стёкла и хром в игре отражают готовую картинку неба, так что грузовики никогда не отражают то, что вокруг. Карточка заставляет их отражать то, что на экране: дорогу, деревья и другие грузовики.

ЧТО ВЫ ЗАМЕТИТЕ

Краска и окна отражают окружение, сильнее всего под пологим углом, а в хроме видна земля.

ЧЕГО СТОИТ

От 0.3 до 0.4 мс на кадр в 4K с reflection pass.

НАСТРОЙКИ

Method: reflection pass трассирует отражения всего экрана за один раз (в SnowRunner Shadows), размывает их по шероховатости каждой поверхности и стабилизирует по кадрам; окна и всё, что проход пропустил, возвращаются к маршу. March only позволяет каждому материалу самостоятельно искать в прошлом кадре: это старый метод, оставленный как вариант.

ПОЛЕЗНО ЗНАТЬ

Отразиться может только то, что есть на экране, поэтому вместо того, что за камерой, по-прежнему видна картинка неба. В игре F6 выключает и включает reflection pass.

АВТОРЫ

Метод reflection pass основан на AMD FidelityFX SSSR (MIT), с трассировкой лучей в экранном пространстве Morgan McGuire и Michael Mara, сэмплированием видимых нормалей Eric Heitz и variance clipping Marco Salvi.

Headlights in reflections

Reflections · включена по умолчанию · требует Object reflections · подробно: Headlights in reflections

Ночью краска, которой на экране нечего отразить, показывает тёплый свет фар.

ЧТО ДЕЛАЕТ

С Object reflections капот или крыша, чьё отражение ничего не находит на экране, показывает картинку неба. Ночью карточка добавляет в эти отражения тёплый свет фар там, где отражённый луч пересекает луч фары.

ЧТО ВЫ ЗАМЕТИТЕ

Ночью капоты и крыши ловят отсвет собственных фар грузовика.

ЧЕГО СТОИТ

Ничего, пока фары выключены. С двумя включёнными фарами не больше 0.15 мс на кадр в 4K, даже если бы глянцевая краска заполняла весь экран (замер на RTX 4080), так что в реальном кадре намного меньше.

ПОЛЕЗНО ЗНАТЬ

Требует Object reflections с reflection pass.

Water reflections

Water · включена по умолчанию · требует SnowRunner Shadows · подробно: Вода: отражения, эффект Френеля, цвет глубины, свечение гребней

В реках, грязи, озёрах, морях и лужах отражаются берега, деревья и грузовики, а не только небо; под пологим углом цвет воды уступает место отражению.

Строка	Значения (по умолчанию выделено жирным)
Sun glitter in shade	**the game's** · off in shadow

ЧТО ДЕЛАЕТ

Реки, грязь, озёра, моря и лужи на дорогах отражают то, что есть на экране: берега, деревья и грузовики, а не только небо. Под пологим углом собственный цвет воды уступает место отражению, как у настоящей воды.

ЧТО ВЫ ЗАМЕТИТЕ

Отражения берегов и деревьев в спокойной воде, грузовиков в дорожных лужах.

ЧЕГО СТОИТ

Немного, и только там, где на экране вода.

НАСТРОЙКИ

Sun glitter in shade: игра оставляет часть солнечных бликов на воде, лежащей в тени от солнца, так что солнечная дорожка может тянуться и под скалой или мостом. Off in shadow гасит их там, где собственная проверка теней воды говорит, что солнце скрыто, в 64 шейдерах озёр и 8 шейдерах рек, которые рисуются с солнечной картой теней. По умолчанию выключено и экспериментально: там, где это пробовали, проверка игры считала воду освещённой, и ничего не менялось.

ПОЛЕЗНО ЗНАТЬ

Требует SnowRunner Shadows для изображения сцены. Для того, что за пределами экрана, остаётся собственное отражение игры.

Wet ground reflections

Reflections · включена по умолчанию · требует SnowRunner Shadows · подробно: Отражения на мокрой земле и ограничение бликов фар

Мокрая грязь и глянцевая земля отражают окружающую сцену: чётко на блестящих участках, мягко на шероховатой грязи.

ЧТО ДЕЛАЕТ

Мокрая земля в игре отражает только готовую картинку неба. Карточка позволяет ей отражать то, что действительно вокруг: грузовики, деревья и берега, из сцены, которую передаёт SnowRunner Shadows. Чётко там, где земля глянцевая, размыто там, где шероховатая, как отражает мокрая грязь. Шероховатую сухую землю карточка не трогает.

ЧТО ВЫ ЗАМЕТИТЕ

Грузовики и деревья мягко отражаются в мокрой грязи и на блестящих мокрых участках, сильнее всего под пологим углом.

ЧЕГО СТОИТ

Немного частоты кадров там, где на экране много мокрой земли: каждый глянцевый пиксель ищет отражение по сцене.

ПОЛЕЗНО ЗНАТЬ

Требует SnowRunner Shadows. Лужи на дорогах это вода, за них отвечает Water reflections. Для того, что за пределами экрана, остаётся картинка неба. В игре F9 выключает и включает её.

River depth colour

Water · включена по умолчанию · подробно: Вода: отражения, эффект Френеля, цвет глубины, свечение гребней

Глубокая вода окрашивается своим цветом в бирюзовый или бурый, а не выцветает в серый.

ЧТО ДЕЛАЕТ

Глубокая речная вода в игре выцветает в ровный серый. Карточка окрашивает её собственным цветом по мере глубины: в бирюзовый в чистых реках, в бурый в мутных.

ЧЕГО СТОИТ

Совсем немного.

Sun glow through waves

Water · включена по умолчанию · требует Water reflections · подробно: Вода: отражения, эффект Френеля, цвет глубины, свечение гребней

Речные волны между вами и низким солнцем светятся цветом самой воды там, где солнце действительно до них достаёт.

ЧТО ДЕЛАЕТ

Если смотреть через воду на низкое солнце, свет проходит сквозь тонкие гребни волн и светится цветом воды: бирюзовым в чистой, бурым в мутной. Карточка добавляет этот свет рекам, сильнее всего при взгляде прямо на свет, и только там, где солнце действительно достаёт до воды: она читает собственные солнечные тени игры, так что вода под деревьями остаётся тёмной.

ЧТО ВЫ ЗАМЕТИТЕ

Светящиеся гребни волн на реках против низкого солнца.

ЧЕГО СТОИТ

Немного: ещё одна проверка солнечной тени на речной воде.

ПОЛЕЗНО ЗНАТЬ

Реки, чей шейдер знает, куда падает тень от солнца (12 из 20 речных шейдеров игры); озёра и моря пока нет. Требует Water reflections.

Headlight glare cap

Water · выключена по умолчанию · подробно: Отражения на мокрой земле и ограничение бликов фар

Блеск фар на мокрой земле и в лужах спокойнее, без длинной яркой полосы поперёк.

ЧТО ДЕЛАЕТ

Ночью фары рисуют на мокрой земле и лужах длинную, очень яркую полосу. Карточка ограничивает этот блик: мокрая земля по-прежнему блестит, но без полосы.

ЧЕГО СТОИТ

Ничего.

ПОЛЕЗНО ЗНАТЬ

Пока по умолчанию выключена: её настраивали до того, как изменились отражения. Отметьте её, если полоса вам мешает.

Reflection cubes

Water · включена по умолчанию · подробно: Кубические карты отражений из фотографий

Нарисованные небеса, которые отражают краска, стёкла и хром и из которых Sky ambient и Bounce light берут цвет неба и земли, сделаны заново по фотографиям: 25 кубмап из 21 панорамы, для ясных и непогожих дней, сумерек, рассвета и ночи каждого региона. Каждая сохраняет яркость игровой кубмапы, а в снежных регионах на земле снег.

ЧТО ДЕЛАЕТ

Каждое состояние дня называет кубическую карту отражений (кубмапу): маленькое нарисованное небо с землёй, которое отражают краска, стёкла и хром там, где в сцене нет ничего ближе, и из которого Sky ambient и Bounce light берут цвет неба и земли. Состояния игры называют 26 таких кубмап в пяти семействах, чьи файлы названы в честь Мичигана, Таймыра, Мексики, Аляски и Скандинавии. Карточка заменяет 25 из них кубмапами, сделанными по фотографиям: ясные дни, непогожие дни, сумерки, рассвет и ночь. Небо, небо прямо над головой и земля выставлены на яркость игровой кубмапы, а в сумерках, на рассвете и ночью ещё и на её цвета, так что каждый час сохраняет свой свет. В двух снежных семействах земля днём, в сумерках и на рассвете покрыта снегом его собственной яркости. Их ночные кубмапы это зимние дневные снимки, переведённые цветокоррекцией в ночь. Гараж сохраняет игровую кубмапу, на которой виден сам гараж. Пока кубмапы установлены, все ясные дни Кольского полуострова и Юкона называют снежную кубмапу, а те немногие состояния Висконсина и Северной Каролины, которые называли снежную, называют кубмапу своего региона.

ЧТО ВЫ ЗАМЕТИТЕ

Хром и стекло отражают сфотографированные небо и землю, а не размытую картинку, на каждой карте и в любой час, и тень с переотражённым светом берут цвет из них.

ЧЕГО СТОИТ

В игре ничего: кубмапы того же размера, что и игровые. Архив больше на 12 МБ.

ПОЛЕЗНО ЗНАТЬ

Меняет boot.pak вместе с Photo grade, Sharper smoke and particles и Night sky и сохраняет их изменения. Кубмапы сделаны с помощью engine\tools\make_cube.js из 21 панорамы CC0 от Poly Haven. В THIRD_PARTY_NOTICES.md перечислены они и их авторы.

Atmosphere and image

Volumetric fog

Atmosphere · выключена по умолчанию · подробно: Объёмный туман и солнечные лучи

Без полос и со свечением в сторону солнца. Можно включить Sun shafts, солнечные лучи сквозь деревья и пыль: они экспериментальные и видны также поверх вашего грузовика и других предметов рядом с камерой.

Строка	Значения (по умолчанию выделено жирным)
Sun shafts	**off** · on

ЧТО ДЕЛАЕТ

Игра рисует туман ступенями, которые видны как полосы. Карточка разбивает эти полосы и добавляет свечение в сторону солнца. Sun shafts: туман также видит солнечные тени, так что сквозь деревья и пыль видны лучи света.

ЧТО ВЫ ЗАМЕТИТЕ

Ровный туман и яркая дымка в сторону солнца. С включёнными солнечными лучами, в тумане и при низком солнце за деревьями: лучи света.

ЧЕГО СТОИТ

Немного.

ПОЛЕЗНО ЗНАТЬ

По умолчанию выключена. Sun shafts на карточке тоже выключены, пока вы их не включите. Они экспериментальные: туман рисуется, не зная, что стоит перед ним, поэтому при низком солнце лучи видны и поверх вашего грузовика и других предметов рядом с камерой. Лучам нужны туман и низкое солнце. В ясный день смотреть почти не на что.

Weather

Atmosphere · включена по умолчанию · подробно: Погода, мусор и брызги

Больше собственной погоды игры: движущиеся тени облаков на всех картах, дожди, которые то усиливаются, то стихают, морось вечером и ночью, облака на горизонте, дождь, видимый дальше (по умолчанию выключен), светлячки ночью и пыльца днём.

Строка	Значения (по умолчанию выделено жирным)
Cloud shadows	**on** · off
Showers	**swell and ease** · as they are
Evening drizzle	**on** · off
Horizon clouds	**on** · off
Far rain	on · **off**
Fireflies	**on** · off
Pollen	**on** · off
Drifting clouds	**on** · off
Far land	**on** · off

ЧТО ДЕЛАЕТ

Игра проводит каждый регион по циклу состояний дня, которые могут нести дождь или снег, а на двух картах ещё и движущиеся тени облаков. Карточка добавляет всего этого больше из собственной погодной системы игры. Cloud shadows: тени облаков появляются на каждой карте, они рисуются из собственных текстур теней игры. Showers: дождь и снег со временем то усиливаются, то стихают, а не держат одну силу, и никогда не бывают сильнее игровых. Evening drizzle: в дождливых регионах идёт слабый дождь в сумерках, ночью и на рассвете, так же как в снежных регионах в это время уже идёт лёгкий снег. Horizon clouds: слой облаков на горизонте, который есть в 17 файлах неба, но выключен, теперь включён. Far rain, по умолчанию выключено: дождь рисуется в 1.5 раза дальше, на 48 м вместо 32. Снег остаётся на игровом расстоянии: когда его рисовали так далеко, сильный снегопад отнимал слишком много частоты кадров. Fireflies: светлячки ночью в дождливых регионах, нарисованные собственным спрайтом светлячка из игры. Pollen: пыльца, плывущая в воздухе в ясные дни тех же регионов. Это два типа погоды из более позднего движка игры, вписанные как два новых файла. Drifting clouds: небо состоит из вращающихся слоёв, у каждого своя скорость, заданная состоянием дня, и в большинстве состояний ближняя полоса облаков и некоторые картинки неба стоят на месте, пока полосы за ними движутся. Там, где картинка облаков стояла на месте, она теперь медленно вращается: около 150 настроек в 17 файлах неба. Звёзды, солнце и луна, северное сияние, Млечный Путь и нарисованный закат остаются на месте. Far land: 46 состояний, в основном на Кольском полуострове, Юконе, в Квебеке и Скандинавии, перестают рисовать землю на 800 м, хотя их дальний туман доходит дальше, так что в ясную погоду земля обрывается слишком близко. Там земля рисуется так далеко, как доходит туман, но не дальше 3000 м.

ЧТО ВЫ ЗАМЕТИТЕ

Тени облаков плывут над каждой картой, а дождь и снег приходят и уходят.

ЧЕГО СТОИТ

Немного. Тени облаков рисуются так же, как игра рисует их на картах, где они есть; остальные части меняют числа, которые игра и так читает. Far rain рисует больше капель вокруг камеры,

поэтому в сильный дождь немного снижает частоту кадров.

НАСТРОЙКИ

Каждая часть включается отдельно, все включены по умолчанию, кроме Far rain. Far land рисует больше земли; во что это обходится, ещё не измерено. Если включены и Fireflies, и Evening drizzle, ночи в дождливых регионах достаются светлячкам, а морось идёт только в сумерках и на рассвете.

ПОЛЕЗНО ЗНАТЬ

Меняет initial.pak вместе с Grass reach, Fill light и Brighter stars и сохраняет их изменения. Другие моды, которые меняют initial.pak, меняют тот же файл: Apply спрашивает, прежде чем собирать поверх него.

Map days

Atmosphere · включена по умолчанию · подробно: Дни карт (Map days), Список загрузки: как пак сообщает игре, что в нём лежит

В четырёх регионах нет своих дней, и они показывают дни другого региона. Мэн показывает дни двух снежных регионов: снег в воздухе и зимний свет на карте без снега. Карточка даёт каждой их карте подходящие дни, подобранные для каждой карты отдельно (map by map), со строкой для каждого региона.

Строка	Значения (по умолчанию выделено жирным)
Maine	Michigan's days · the game's · **map by map**
British Columbia	North Carolina's days · the game's · **map by map**
Austria	both maps alike · the game's · **map by map**
Washington	Michigan's autumn · the game's · **map by map**

ЧТО ДЕЛАЕТ

В файле уровня каждой карты названы состояния дня, по которым она проходит, и её файл неба. У четырёх регионов своих нет, и они называют чужие. Мэн: его первая карта называет состояния Юкона, а вторая Аляски, двух снежных регионов, поэтому в Мэне идёт их снег в сумерках и ночью и стоит их свет. Карточка даёт обеим картам дни, погоду и небо Мичигана. Британская Колумбия: её карты называют дни Мичигана, а строка даёт им дни Северной Каролины, которые сама игра даёт Вашингтону, соседнему региону. Австрия: первая карта называет дни не того региона, что вторая, и строка даёт первой дни второй. Вашингтон, осенний регион: три его карты называют лето Северной Каролины, а строка даёт им осень Мичигана.

ЧТО ВЫ ЗАМЕТИТЕ

В Мэне: никакого снега в воздухе, а свет, погода и небо осенней лесной карты.

ЧЕГО СТОИТ

Ничего.

НАСТРОЙКИ

Строка для каждого региона, у каждой по умолчанию map by map: дни подобраны для каждой карты отдельно. Остальные значения дают картам региона дни одного другого региона или оставляют игровые. Maine: на The Lowland нет снега на земле, и она получает дождливые дни Мичигана; Yellowrock National Forest со снегом и льдом получает весну Юкона с лёгким снегом вместо метелей. Оба набора собственные, мэнские, так что позже каждую карту можно будет настроить отдельно. British Columbia: затопленная Duncan Bay получает лето Висконсина с его сильными дождями; North Peak National Park получает дни Северной Каролины. Austria: Unterfeld, сельскохозяйственная карта, получает лето Теннесси; Foothills сохраняет игровую Северную Каролину. Washington это осенний регион, три карты которого показывают лето Северной Каролины: его строка даёт им Michigan's autumn, а при map by map Selten River Mountains получает горную осень Зурдании.

ПОЛЕЗНО ЗНАТЬ

Карточка меняет несколько названий прямо в ваших файлах уровней (level_us_06_01.pak и подобных), а Restore записывает обратно игровые названия, после чего файлы становятся такими же, как были, байт в байт. Файл уровня, изменённый другим модом, остаётся как есть. Обновление игры или проверка файлов в магазине возвращает игровые названия: снова нажмите Apply. Большинство значений называют только дни, которые уже есть в игре. Строка Maine со значением

map by map также добавляет двадцать небольших файлов в initial.pak и прописывает их в его списке загрузки. Если обновление игры или проверка файлов в магазине заменит initial.pak, а файлы уровней Мэна всё ещё ссылаются на них, Мэн перестанет загружаться, пока Apply не запустят снова.

Smoke glow

Atmosphere · включена по умолчанию · подробно: Дым: свечение, форма, мягкие края

Выхлоп и другой дым светятся, когда солнце позади них.

Строка	Значения (по умолчанию выделено жирным)
More kinds of smoke	**off** · on

ЧТО ДЕЛАЕТ

Выхлоп, дым, пыль и пар светятся, когда солнце позади них, как настоящий дым светится против солнца.

ЧТО ВЫ ЗАМЕТИТЕ

Выхлоп и облака пыли загораются, когда вы смотрите в сторону низкого солнца.

ЧЕГО СТОИТ

Совсем немного.

НАСТРОЙКИ

Свечение работает в 162 из 205 шейдеров частиц игры. More kinds of smoke добавляет ещё восемь, которые освещают свои клубы так же; по умолчанию выключено, потому что в игре это ещё не проверяли.

Smoke shape and softer edges

Atmosphere · включена по умолчанию · подробно: Дым: свечение, форма, мягкие края

У клубов дыма и пыли появляются освещённая и затенённая стороны, а спрайты мягче растворяются там, где касаются земли.

ЧТО ДЕЛАЕТ

Игра освещает каждый клуб дыма или пыли одним плоским цветом. Карточка попиксельно придаёт объём 80 освещаемым спрайтам клубов: освещённая сторона к солнцу, затенённая дальняя, светлее верх и чуть темнее плотная середина. 110 спрайтов, которые растворяются там, где врезаются в землю или грузовик, теперь растворяются на вдвое большем расстоянии, так что резкой линии там, где клуб встречается с поверхностью, больше нет.

ЧТО ВЫ ЗАМЕТИТЕ

Более круглые и плотные облака выхлопа и пыли и никакого резкого края там, где дым касается земли.

ЧЕГО СТОИТ

Немного: несколько лишних инструкций в шейдерах частиц.

ПОЛЕЗНО ЗНАТЬ

Работает и с Smoke glow, и без неё.

Sharper smoke and particles

Atmosphere · включена по умолчанию · подробно: Более чёткий дым и частицы

Игровые текстуры дыма, пыли, брызг и искр в удвоенном размере, чтобы частицы оставались чёткими вблизи.

ЧТО ДЕЛАЕТ

Дым, пыль, брызги, искры, листья и другие частицы рисуются из маленьких текстур, которые игра растягивает, так что вблизи они выглядят мыльными и квадратными. Карточка ставит те же текстуры в удвоенном размере, увеличенные из игровых с помощью NVIDIA DLSS 5 Visual Enhancer, в boot.pak рядом с цветокоррекцией.

ЧТО ВЫ ЗАМЕТИТЕ

Чётче выхлоп, облака пыли и брызги рядом с камерой, с теми же цветами и формами.

ЧЕГО СТОИТ

Видеопамять под увеличенные текстуры. Из-за этого набора установщик больше на 114 МБ.

ПОЛЕЗНО ЗНАТЬ

Меняет boot.pak вместе с Photo grade и Night sky и сохраняет их изменения. Другие моды, которые меняют boot.pak, например текстурпак, тоже сохраняются: Apply спрашивает, прежде чем собирать поверх них.

Debris and splashes

Atmosphere · включена по умолчанию · подробно: Погода, мусор и брызги

Листья, трава, ветки и грязь из-под колёс полминуты лежат на земле, и шины их толкают, а не исчезают через четыре секунды. Брызги грязи, воды и снега дают больше капель, которые дольше лежат и оставляют больше следов на грузовике. Снежная пыль висит в воздухе. Выхлоп и пыль тоже держатся дольше.

Строка	Значения (по умолчанию выделено жирным)
Splash drops	**4x** · 8x

ЧТО ДЕЛАЕТ

То, что выбрасывают колёса, пропадает за несколько секунд: листья, трава и ветки через четыре, капли брызг грязи или воды через две. Карточка оставляет их подольше. Листья, трава, ветки, комья земли и камни живут от 30 до 40 секунд и ложатся на землю, где их толкают шины. Брызги грязи, воды и снега дают вчетверо больше капель, капли лежат вдвое дольше, сами брызги держатся вдвое дольше и оставляют на грузовике вдвое больше следов грязи и воды. Выхлоп, пыль и пар висят в воздухе в полтора раза дольше. Снежная пыль висит в воздухе, как в жизни: снежные частицы падают с половинной гравитацией и меньшим сопротивлением воздуха и живут 15 секунд, а снежные брызги держатся втрое дольше. Брызги, которые бросает колесо, буксующее в воде, летят примерно в 2.5 раза дальше и в полтора раза крупнее: у них и их капель меньше сопротивление воздуха, и живут они дольше. Во всём остальном сопротивление воздуха, гравитация, ветер и размеры остаются игровыми.

ЧТО ВЫ ЗАМЕТИТЕ

За грузовиком остаётся след из листьев и вырванной травы, в брызгах грязи больше капель и они живут дольше, от буксующего колеса летят большие фонтаны воды, выхлоп держится в воздухе.

ЧЕГО СТОИТ

Частота кадров там, где из-под колёс летит много всего: одновременно живёт примерно в десять раз больше частиц, а в брызгах вчетверо больше капель. При 8x всё это вдвое больше.

НАСТРОЙКИ

Splash drops: 4x или 8x игровых капель на одни брызги.

ПОЛЕЗНО ЗНАТЬ

Меняет initial.pak вместе с Grass reach, Fill light, Brighter stars и Weather и сохраняет их изменения. Другие моды, которые меняют initial.pak, меняют тот же файл: Apply спрашивает, прежде чем собирать поверх него.

Night sky

Atmosphere · включена по умолчанию · подробно: Ночное небо, яркие звёзды и небо

Звёздная карта NASA вместо игровой текстуры звёзд: звёзды настоящего неба тонкими точками и свечение Млечного Пути.

ЧТО ДЕЛАЕТ

Ночное небо игры это маленькая текстура, где каждая звезда занимает один тексель. Карточка ставит на её место звёздную карту NASA с вчетверо большим числом пикселей: звёзды настоящего неба тонкими точками и свечение Млечного Пути. В Скандинавии, на Кольском полуострове и в Квебеке ночное небо это фотография с северным сиянием. Там те же звёзды нарисованы прямо на фотографии.

ЧТО ВЫ ЗАМЕТИТЕ

Ночное небо, полное тонких звёзд, и Млечный Путь через него.

ЧЕГО СТОИТ

Видеопамять под увеличенные текстуры неба. Из-за этого набора установщик больше на 65 МБ.

ПОЛЕЗНО ЗНАТЬ

Меняет boot.pak вместе с Photo grade и Sharper smoke and particles и сохраняет их изменения, а ещё выставляет в initial.pak уровни, нужные её небу-фотографиям. Новые звёзды тоньше игровых и при игровом уровне выходят тусклыми. Brighter stars настроена под них.

АВТОРЫ

Звёздная карта сделана из Deep Star Maps 2020. Источник: NASA/Goddard Space Flight Center Scientific Visualization Studio. Gaia DR2: ESA/Gaia/DPAC.

Skies

Atmosphere · включена по умолчанию · подробно: Ночное небо, яркие звёзды и небо

Нарисованные небеса всех регионов сделаны заново по фотографиям настоящего неба: 49 игровых небесных куполов, полос облаков и ночных слоёв, каждый со своим размером и яркостью; солнце с фотографий убрано, так что единственное солнце здесь игровое. Северное сияние зелёное, как настоящее. Облака по-прежнему плывут.

ЧТО ДЕЛАЕТ

Небо региона это стопка слоёв: нарисованный купол всего верхнего неба, полосы облаков, плывущие над ним, звёзды, солнце. Карточка заменяет 49 игровых небесных куполов и полос облаков во всех регионах на сделанные по фотографиям настоящего неба, Млечный Путь на двух ночных слоях на настоящий из звёздной карты NASA, а синее и фиолетовое северное сияние на зелёное, как у настоящего. Каждая текстура сохраняет размер той, которую заменяет, и игровую яркость для неё. Солнце с фотографий убрано, так что солнечный диск игры остаётся единственным солнцем; полосы облаков вырезаны из фотографий с их собственной прозрачностью.

ЧТО ВЫ ЗАМЕТИТЕ

Настоящие облака и небо днём, которые по-прежнему плывут; в сумерках, на рассвете и ночью поверх них игровой оттенок.

ЧЕГО СТОИТ

В игре ничего. Текстуры идут отдельной загрузкой, набором неба (SnowRunnerNextGen_Skies.zip, около 100 МБ, 300 МБ в распакованном виде), который распаковывается в папку этой программы; без него Apply пропускает небо и сообщает об этом.

ПОЛЕЗНО ЗНАТЬ

Радуга, комета, диски и ореолы солнца и луны остаются игровыми. Меняет boot.pak вместе с Photo grade, Sharper smoke and particles, Night sky и Reflection cubes и сохраняет их изменения. Небо сделано с помощью engine\tools\make_cube.js и make_clouds.js из 23 панорам CC0 от Poly Haven, Млечный Путь из Deep Star Maps 2020 от NASA.

Brighter stars

Atmosphere · включена по умолчанию · подробно: Ночное небо, яркие звёзды и небо

Звёзды ночного неба ярче и лучше видны на тёмном фоне.

Строка	Значения (по умолчанию выделено жирным)
Brightness	**3x** · 2x

ЧТО ДЕЛАЕТ

Ночное небо рисует звёзды одним слоем, с низким уровнем, заданным для каждого региона. Карточка поднимает этот уровень ночью во всех регионах, где звёзды идут отдельным слоем, чтобы они выделялись на тёмном небе. Сумерки и рассвет сохраняют игровой уровень.

ЧТО ВЫ ЗАМЕТИТЕ

Ночное небо с хорошо заметными звёздами.

ЧЕГО СТОИТ

Ничего: карточка меняет только яркость слоя, который игра и так рисует.

НАСТРОЙКИ

Brightness: в три или в два раза выше игрового уровня.

ПОЛЕЗНО ЗНАТЬ

Меняет initial.pak вместе с Grass reach и Fill light и сохраняет их изменения. Трёхкратный уровень подобран под звёздную карту Night sky, чьи тонкие звёзды при игровом уровне выходят тусклыми. С игровой текстурой звёзд звёзды просто становятся во столько же раз ярче. Регионы, где ночное небо это фотография, карточка не меняет: с Night sky звёзды на их фотографиях имеют постоянный уровень.

Tonemap

Image · включена по умолчанию · подробно: Тонмаппинг, блум и цветокоррекция

Сильнее контраст и слабее блум.

ЧТО ДЕЛАЕТ

Тонмаппинг переводит свет сцены в цвета вашего экрана. Карточка использует кривую с большим контрастом, чем у игры. Средний серый остаётся на месте, тень становится глубже и сохраняет детали, а яркие места становятся ярче и плавно уходят в белое без жёсткой обрезки. Блум ослаблен до 60 %.

ЧТО ВЫ ЗАМЕТИТЕ

Глубже тени и насыщеннее цвет, меньше блёклости.

ЧЕГО СТОИТ

Практически ничего.

Bloom soft cut

Image · включена по умолчанию · подробно: Тонмаппинг, блум и цветокоррекция

Никакой дымки от яркого снега и неба.

ЧТО ДЕЛАЕТ

Блум заставляет яркое светиться. Игровой блум включается рано, поэтому яркий снег и небо стелют дымку по всей картинке. Карточка даёт ему мягкий порог: светится только действительно яркое.

ЧТО ВЫ ЗАМЕТИТЕ

Чистые снежные виды без молочной дымки; фонари и солнце по-прежнему светятся.

ЧЕГО СТОИТ

Практически ничего.

Photo grade

Image · включена по умолчанию · подробно: Тонмаппинг, блум и цветокоррекция

Цвет ближе к фотографии: зелень, осенняя листва и фиолетовая дымка спокойнее, белое чуть темнее, нет грязи на объективе поверх неба. Встроено в собственную цветокоррекцию игры.

Строка	Значения (по умолчанию выделено жирным)
Strength	**100 %** · 50 %

ЧТО ДЕЛАЕТ

Игра завершает каждую картинку цветокоррекцией под время суток. Карточка встраивает в неё фотографическую цветокоррекцию: зелень, осенняя листва и фиолетовая дымка становятся спокойнее, а белое ложится чуть ниже и не слепит. Сколько убрать у каждого цвета, измерено по снимкам тех же сцен с DLSS 5. Серое остаётся серым, а собственная цветокоррекция игры остаётся под ней. Ещё карточка убирает игровую грязь на объективе: мягкие круги, которые камера добавляет поверх яркого неба и облаков.

ЧТО ВЫ ЗАМЕТИТЕ

Листва и холмы в более спокойных цветах фотографии, дымка серо-голубая, а не лавандовая, меньше блеска на белой краске и снегу и никаких размытых кругов поверх облаков.

ЧЕГО СТОИТ

Ничего: игра и так делает цветокоррекцию картинки.

НАСТРОЙКИ

Strength: 100 % это измеренная цветокоррекция, 50 % её половина.

ПОЛЕЗНО ЗНАТЬ

Меняет четыре цветовые таблицы, которые использует освещение дня, сумерек и рассвета (их делят и несколько ночных сцен), внутри boot.pak, куда идут также Sharper smoke and particles и Night sky; гараж, карта и самая тёмная ночная таблица сохраняют свои. Лучше всего вместе с Fill light, которая отвечает за тень, а эта карточка за цвет. Если другой мод добавил свои файлы в boot.pak (например, текстурпак), Apply спрашивает, прежде чем собирать поверх них, и они остаются (см. How it works, раздел Other mods).

Scenery

Scenery detail

Scenery · включена по умолчанию · подробно: Детализация окружения и дальность травы

Объекты окружения дольше сохраняют полную детализацию, так что на ходу меньше что-то меняется на глазах. All meshes делает shared.pak больше 2 ГБ. Shown far away: при plants растения (деревья, кусты, камни, ветки), при everything вообще всё больше не исчезает на заданном расстоянии. Small shadows: то, что не отбрасывает тень или теряет её в нескольких метрах, получает тень.

Строка	Значения (по умолчанию выделено жирным)
Set	nature · **all meshes**
Shown far away	as the game · plants · **everything**
Small shadows	off · plants · **everything**

ЧТО ДЕЛАЕТ

Камни, деревья и кусты переключаются на более простые модели рядом с вами (некоторые камни уже на 10 м), и это видно: формы и тени скачут на ходу. Карточка отодвигает первое переключение минимум на 40 м, а следующие на 70, 110 и 160.

ЧТО ВЫ ЗАМЕТИТЕ

Камни и деревья гораздо меньше меняются на глазах.

ЧЕГО СТОИТ

Более детальные модели рисуются дальше: немного частоты кадров в густом лесу. Каждое из двух дополнений добавляет к этому: больше растений вдали, больше теней.

НАСТРОЙКИ

Set: nature меняет камни, деревья и кусты (351 модель). All meshes меняет все модели со ступенями детализации (1626), из-за чего shared.pak становится больше 2 ГБ; игра загружает его нормально. Shown far away: игра перестаёт рисовать большинство объектов дальше их собственного расстояния (мелкие камни и ветки от 55 до 80 м, деревья и здания дальше), и видно, как впереди появляются предметы. Plants убирает это расстояние у всех растений (314 моделей: деревья, кусты, камни, ветки, корни), everything у всех моделей, где оно есть (3468: ещё и знаки, заборы, трубы, дома и всё остальное). Small shadows: некоторые модели вообще не отбрасывают тень или теряют её на расстоянии от 25 до 50 м. Plants даёт каждому из этих растений тень на любом расстоянии (163 модели), everything делает то же для всех 395.

ПОЛЕЗНО ЗНАТЬ

Обновление игры или проверка файлов в Steam возвращает оригинальный shared.pak. После этого снова нажмите Apply.

Grass reach

Scenery · включена по умолчанию · подробно: Детализация окружения и дальность травы

Трава видна дальше. Снижает частоту кадров там, где много травы.

Строка	Значения (по умолчанию выделено жирным)
Reach	3x · **2x**

ЧТО ДЕЛАЕТ

Большая часть травы исчезает примерно в 30 м от камеры. Карточка рисует её до 60 м, и она по-прежнему плавно вырастает по мере приближения.

ЧЕГО СТОИТ

Частота кадров на травянистых участках: трава покрывает до четырёх раз больше земли, а на 3x она остаётся густой дальше, чем на 2x.

НАСТРОЙКИ

Reach: 3x или 2x игровых расстояний исчезновения, каждая ступень не дальше 50, 55 и 60 м.

ПОЛЕЗНО ЗНАТЬ

Игра рисует траву только примерно до 64 м от камеры. Если задать дальше, трава на краю появляется сразу целиком, а не вырастает, поэтому ни одна ступень не уходит дальше 60 м; ступень, которую сама игра задаёт дальше, остаётся как есть. Меняет initial.pak, файл, который меняют и другие моды.

Interface

Next Gen logo

Interface · включена по умолчанию · подробно: Логотип Next Gen и запуск без заставки

NEXT GEN под логотипом игры на титульном экране, первом экране загрузки, в главном меню, меню паузы и на карте.

Строка	Значения (по умолчанию выделено жирным)
Start-up	the game's intro · **Next Gen splash only**

ЧТО ДЕЛАЕТ

Добавляет NEXT GEN под логотип игры везде, где его показывает интерфейс: на титульном экране, первом экране загрузки, в главном меню, меню паузы и на карте.

ЧТО ВЫ ЗАМЕТИТЕ

Название мода под логотипом SnowRunner, чтобы с первого взгляда было видно, что он установлен.

ЧЕГО СТОИТ

В игре ничего. Первый Apply сохраняет копию gfx.pak, несколько сотен МБ, чтобы Restore мог её вернуть.

НАСТРОЙКИ

Start-up: the game's intro (её предупреждение и логотипы издателей, около 18 с) или Next Gen splash only: игра сразу переходит к первому экрану загрузки. Замер 2026-10-17 с прогретым дисковым кэшем: титульный экран появился через 41 с после запуска со вступлением и через 24 или 25 с без него. По умолчанию выбрано Next Gen splash only.

ПОЛЕЗНО ЗНАТЬ

Меняет gfx.pak, файл интерфейса. Строка Start-up меняет стартовый ролик игры в том же файле: это две небольшие правки в его скрипте, а версия игры с другим роликом сохраняет своё вступление (Apply сообщает об этом). У титульного экрана, первого экрана загрузки и главного меню есть своя картинка для логотипа, и они заменяются целиком. В меню паузы и на карте логотип делит лист с другими картинками: там меняется только логотип, так что лист мода интерфейса сохраняет всё остальное, а лист с другой раскладкой остаётся как есть.

Часть 4. Как это устроено, подробно

В этой части объясняется, как работает каждая карточка: метод с самых основ, формулы и числа, на которых он стоит, проходы и буферы, которые он читает и пишет, что патч меняет в игре и зачем, чего это стоит и где метод не дотягивает. Числа те же, что в коде; если число задаёт параметр SnowRunnerShadows.ini, указано его имя.

Как Next Gen попадает в игру

SnowRunner рисует через Direct3D 11. Всё, что видно на экране, создают маленькие программы на видеокарте, шейдеры, которые игра хранит заранее скомпилированными в одном файле, **shader.pak**. Next Gen меняет игру четырьмя способами, и каждая карточка в этой части использует один или несколько из них.

1. Шейдеры, заменённые целиком

Часть шейдеров игры заменена нашими: они написаны на HLSL и скомпилированы компилятором Microsoft (fxc) в тот же формат байт-кода, DXBC. Проход фонового затенения (ambient occlusion; два шейдера, хеши 0xEA2414F8 и 0xA3716E2B) становится GTAO, шейдер тумана (0x871EF8CC) получает джиттер и фазовую функцию, тонмаппинг (0x221304E2) и блум (0x52879E18) получают новые кривые. Внутри shader.pak шейдеры лежат в одной записи, *shadercachedx11.sdc*: 12-байтный заголовок (метка даты и два размера) и один поток zlib. В распакованном виде это список include-файлов, число программ, число блобов, затем все блобы (шейдер DXBC, перед которым записан его размер) и таблица, связывающая программы с блобами по индексу. Шейдеры постобработки лежат во втором кэше, *common_pc_sm50_hdr.sdc*, в том же виде. Заменённый блоб может поменять размер: инструмент переписывает поток и таблицу и снова сжимает их.

2. Вставки в собственные шейдеры игры

Большинство карточек сохраняют шейдер игры и дополняют его. В игре тысячи шейдеров материалов, по одному на каждый вид поверхности (3229 только для техники и объектов), и заменить их все значило бы переписать материалы игры. Вместо этого инструмент-патчер находит в каждом одну инструкцию, **якорь**, и ставит туда вызов нашей вспомогательной функции. Для отражений якорем служит выборка материала из кубической карты неба:

> sample_l r4.xyz, r5.xyz, t86.xyzw, s8, r3.w // кубмапа t86 на мип-уровне r3.w = $\sqrt{\text{roughness}} \times 8 + 0.5$

Вспомогательная функция встаёт на место этой выборки. Она получает то же, что получала выборка (направление и мип-уровень), и то, что ей нужно сверх того (мировую позицию пикселя), а её результат идёт туда же, куда шёл результат выборки, поэтому всё, что игра делает после неё (отражательная способность, член Френеля, множитель g_fReflCubeGGXScale), по-прежнему применяется. Функция компилируется из HLSL один раз; её инструкции копируются в каждый целевой шейдер, а регистры переименовываются в свободные.

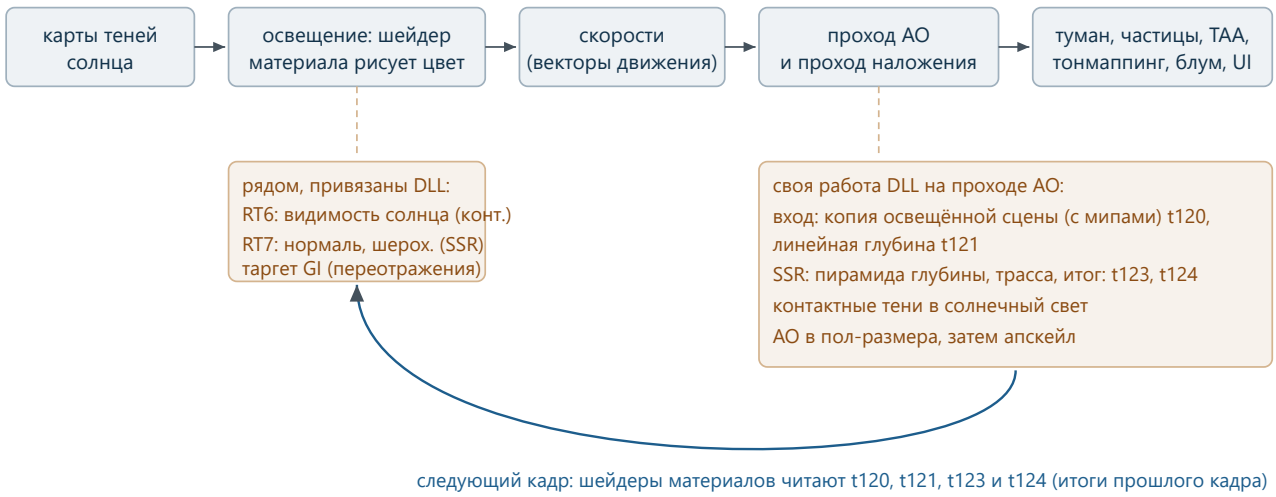
Вставка в скомпилированный код это хирургия над контейнером. Файл DXBC состоит из чанков (программа в SHEX, её входы и выходы, статистика) и контрольной суммы. После вставки инструкций инструмент (dxbc_patch.js) исправляет всё, что зависит от раскладки: размеры контейнера и чанков, смещения чанков, длину программы, число временных регистров (dcl_temps), счётчики инструкций в чанке статистики и контрольную сумму, вариант MD5 с нестандартным последним блоком, проверенный на собственных шейдерах игры. Шейдер остаётся нетронутым, если патч не может доказать, что вставка подходит: ровно один якорь, нужные входы объявлены, после якоря ничто не читает регистр, в который писал только якорь. Каждый инструмент-патчер работает и со стоковым шейдером, и с уже изменённым другой карточкой, потому что карточки трогают разные инструкции.

3. Числа, изменённые на месте

Некоторые карточки меняют только константу: ограничение бликов фар (32 становится 6), декодирование карты освещения (light map), номер кадра во вступительном ролике. Эти патчи сохраняют размер всего, что затрагивают.

4. SnowRunner Shadows, DLL

Некоторым техникам нужны данные, которых нет ни в одном отдельном шейдере, например весь освещённый кадр или пирамида глубины. Игра загружает файл hid.dll из своей папки раньше системной копии, поэтому Next Gen кладёт туда SnowRunner Shadows. Эта DLL передаёт каждый вызов настоящей hid.dll (или DLL другого мода, переименованной в hid_chain.dll) и перехватывает (hook) контекст устройства Direct3D 11. Она следит, какие шейдеры привязывает игра, узнаёт нужные ей проходы по их шейдерам и добавляет свою работу между ними. Она никогда не пишет в общие таблицы методов DXGI, которые перехватывают и другие моды, и оверлей Steam: её диагностические функции (журнал кадра, таймеры GPU, профили) берут swap chain и фабрику игры, и каждый из этих объектов получает собственную копию таблицы методов с перехватом в ней. Клиент Steam от 2026-10-21 ставит свои перехваты поверх тех, что находит в общем Present у DXGI, и перехват там, сохранивший перехват Steam как свою настоящую функцию, вызывал сам себя, пока не кончался стек (это видно в дампе падения у игрока с другим модом). Собственные перехваты Direct3D 11 запоминают для каждой таблицы методов, в которой стоят, заменённые функции и восемь слотов, которые никогда не трогают; если другая программа потом даст устройству или контексту частную копию этой таблицы (перехваты переходят в неё вместе с остальным), копию узнают по этим восьми слотам и дают ей те же функции, и следующий вызов не падает.



Один кадр глазами Next Gen. Проходы игры наверху, то, что добавляет Next Gen, под ними. Результаты проходов DLL доходят до шейдеров материалов на кадр позже, поэтому каждый, кто их читает, делает репроекцию в предыдущий кадр.

Части DLL, на которые опираются карточки:

Что	Где находится	Кто читает
Входное изображение (feed): освещённая сцена прошлого кадра с цепочкой мип-уровней и её линейная глубина	t120 и t121, привязаны к каждому шейдеру, который их объявляет	отражения объектов, вода, мокрая земля, переотражённый свет
Рендер-таргет 7: мировая нормаль × 0.5 + 0.5 и перцептивная шероховатость (RGBA8)	привязан рядом с проходом освещения, в него пишут шейдеры материалов (карточка SnowRunner Shadows вместе с Object reflections)	проход отражений
Результат прохода отражений: яркость (radiance) × уверенность и сама уверенность (RGBA16F), а также движение объектов	t123, t124	шейдеры материалов, в следующем кадре
Рендер-таргет 6: видимость солнца, найденная для каждой принимающей поверхности	привязан рядом с проходом освещения	проход контактных теней
Второй таргет AO: свет, отражённый от ближних поверхностей	в него пишет проход AO	Bounce light

Без DLL каждый шейдер со вставкой всё равно работает: вспомогательная функция, которая находит свои текстуры непривязанными, возвращает ровно то, что вернула бы собственная инструкция игры. F8 в игре выключает и включает всю шейдерную работу Next Gen, чтобы сравнить вид с одного места. Для этого DLL хранит таблицу игровых оригиналов всех изменённых шейдеров (SnowRunnerShadows.stock, около 5 600 штук) и, пока действует F8, рисует оригинал вместо изменённого (F11 делает то же самое только для прохода AO). Оригиналы собираются, когда F8 или F11 впервые их запрашивает, каждый шейдер один раз, и сохраняются: первое нажатие занимает мгновение, а при запуске не собирается ни один. До октября 2026 они собирались при запуске игры: около 5 600 сборок шейдеров до первого кадра, которые ни для какого сравнения не были нужны.

Те же переключатели есть в виде кнопок в оверлее NEXT GEN, меню, общем для модов Next Gen (его открывает F2; в NextGen.ini рядом с exe игры можно назначить другую клавишу). DLL экспортирует список (NgGraphicsGetApi, версия 1): каждый переключатель, функцию которого включил ini, с именем, подсказкой, состоянием и способом его задать, и несколько строк состояния (устройство перехвачено, размер карты теней, стоковые шейдеры, ReShade). Оверлей находит DLL по полному пути в папке игры и рисует вкладку Graphics по этому списку в потоке present игры. Кнопка и её клавиша переключают один и тот же флаг, поэтому никогда не расходятся; изменение держится до закрытия игры, а начальное состояние задаёт SnowRunnerShadows.ini. Пока в меню NEXT GEN открыта вкладка Graphics, клавиши от F4 до F11 молчат, потому что другие моды используют часть тех же клавиш (KeysWithMenu=1 в ini оставляет их рабочими); с одним Next Gen Graphics они работают как раньше. Каждый мод Next Gen пишет в слоты импорта игры под одной блокировкой (Local\NextGen.Slots.<pid>) через compare-exchange, который сохраняет прежнее содержимое как оригинал, поэтому два мода, перехватывающие D3D11CreateDevice, встают в цепочку в любом порядке.

Солнечные тени: разрешение, края, мягкость, швы

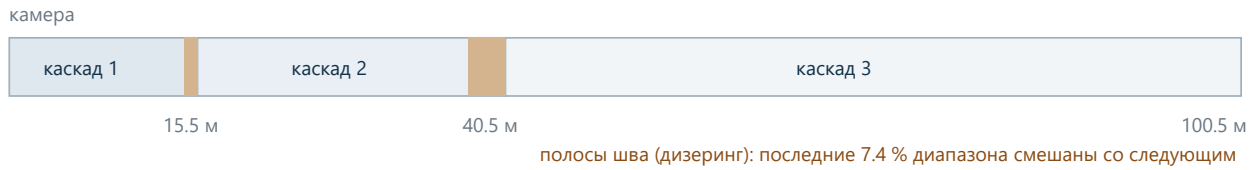
Карточки: SnowRunner Shadows (Resolution, Self-shadow fix), Shadow edges, Blocker search. Четыре изменения в том, как игра рисует и фильтрует солнечную тень.

Карты теней в одном абзаце

Перед кадром игра рисует сцену со стороны солнца в текстуру глубины, карту теней (shadow map): каждый тексель хранит расстояние от солнца до ближайшей поверхности вдоль этой линии. Когда считается освещение пикселя кадра, его позиция проецируется в эту текстуру; если сохранённое расстояние d_{map} меньше собственного расстояния пикселя d_{recv}, между ним и солнцем что-то есть:

> $$\mathrm{lit} = [\, d_{recv} - \mathrm{bias} \le d_{map}(u, v) \,]$$

Один такой тест даёт жёсткий край, где тексели текстуры видны лесенкой. Фильтры усредняют много тестов вокруг пикселя (percentage-closer filtering, PCF), и край смягчается. SnowRunner держит три каскада (cascaded shadow maps), по текстуре на ближний, средний и дальний диапазон, бок о бок в одной текстуре глубины 4608 × 1536 на максимальной настройке (1536 × 1536 на каскад). Пиксель берёт первый каскад, граница которого лежит дальше него: 15.5 м, 40.5 м, 100.5 м от камеры.



Три каскада вдоль глубины обзора. Тексели каждого следующего каскада покрывают от 2 до 2.5 раз больше земли.

Resolution: текстура теней побольше (SnowRunner Shadows, строка Resolution)

Размер текстуры теней не задаётся ни в одном файле игры: он берётся из исполняемого файла. DLL узнаёт текстуру, когда игра её создаёт (текстура глубины втрое шире своей высоты), создаёт её в Factor раз больше и, пока она привязана как цель глубины, масштабирует на тот же множитель каждый вьюпорт и прямоугольник отсечения (scissor), который задаёт игра. Шейдеры читают её в нормализованных координатах (смещения каскадов 0, 1/3, 2/3), поэтому их менять не нужно. Factor 3.5 это наибольший, который помещается: 4608 × 3.5 = 16128, чуть меньше предела Direct3D 11 в 16384. Тексель тогда покрывает $1/\mathrm{Factor}^2$ прежней площади. Цена: при Factor 3.5 примерно на 7 мс на кадр больше при 3840 × 2160. По умолчанию на карточке стоит 1x, потому что перестроенные края (значение rebuilt edges, о нём ниже) дают ровные тени и при родном размере игры.

Self-shadow fix (строка Self-shadow fix)

Чтобы поверхность не затеняла сама себя («акне» теней, shadow acne), игра рисует текстуру теней со смещением глубины по наклону (slope-scaled bias): глубина каждого примитива отодвигается пропорционально её наклону на протяжении *одного текселя*. В текстуре, которая в Factor раз детальнее, тексели в Factor раз меньше, поэтому та же настройка даёт в Factor раз меньшее смещение, а фильтры дотягиваются так же далеко, как раньше. Поверхности, на которые солнце светит по касательной, тогда начинают затенять сами себя, и в каждом каскаде по-разному (теневая сторона валуна на закате переключается на границе 15 м). Когда исправление включено, DLL на время отрисовки масштабированной текстуры подменяет состояния растеризатора игры копиями, в

которых смещение по наклону умножено на Factor. Каждый примитив получает ровно то смещение, что и при исходном размере.

Фильтр игры и откуда берётся лесенка

Путь теней высокого качества в игре (1269 шейдеров материалов: грузовики, объекты, ландшафт) делает три шага: выбирает каскад, ищет блокеры, чтобы оценить, насколько мягким должен быть край (размер полутени), затем фильтрует 16 выборками сравнения в фиксированных точках диска Пуассона, растянутых на размер полутени. Заметны две вещи:

- При Factor от 1 до 2 один тексель рядом с камерой покрывает от 2 до 16 пикселей экрана, а контактная полутень игры около половины текселя. Любой фильтр, достаточно узкий, чтобы контактные тени оставались чёткими, показывает лесенку текселей.
- Когда полутень охватывает больше текселей, чем покрывают 16 выборок, каждая выборка добавляет свою ступенчатую копию края.

Shadow edges, чёткий вариант: треугольный фильтр по сетке

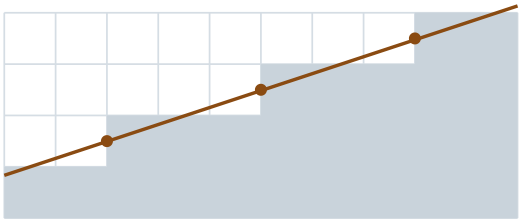
Первый фильтр на замену (hq_grid) сохраняет выбор каскада, поиск блокеров и полутень игры и заменяет только 16 выборок. Он берёт треугольный фильтр (tent: линейный спад, полуширина H = 1.6 единицы ядра, под разброс стоковых выборок) по сетке билинейных выборок сравнения, шаг которой меряется в настоящих текселях, а выборок по каждой оси столько, сколько нужно для этой ширины (от 2 до 8). Билинейная выборка сравнения уже возвращает освещённую долю четырёх текселей, взвешенную по положению, поэтому сетка интегрирует треугольник по текселям:

> $\mathrm{lit} = \Sigma_{i,j}\, w_i\, w_j \cdot \mathrm{cmp}(u_i, v_j), \;\; w_i = \max(0, 1 - |x_i| \,/\, H) \,/\, \Sigma_k \max(0, 1 - |x_k| \,/\, H)$

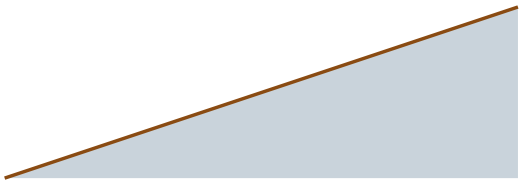
Шум не используется, поэтому для чистой картинки TAA не нужен.

Shadow edges, по умолчанию: ревекторизация

Фильтр по умолчанию (hq_revec, написан по работам Macedo и Apolinário, «Revectorization-Based Shadow Mapping», GI 2016, и Bondarev, «Shadow Map Silhouette Revectorization», I3D 2014) восстанавливает прямой край, который тексели лишь дискретизируют.



тесты текселей (серые: тень) и прямая через середины ступенек



край по этой прямой, смягчённый треугольником поперёк

Ревекторизация: от лесенки тестов текселей к прямому краю, который они дискретизируют.

1. Четыре инструкции GatherCmp читают тесты глубины 4 × 4 текселей вокруг пикселя. Все одинаковы: пиксель просто освещён или в тени, готово (так у большинства пикселей).
2. Главное направление края: если изменений между строками больше, чем между столбцами, край в основном горизонтальный; дальше всё считается в этой системе координат. Затем ищется изменение, ближайшее к пикселю в его собственном столбце, между строками R и R + 1.
3. Край прослеживается в обе стороны, по два столбца на GatherCmp, до 8 текселей в каждую сторону и в пределах этого каскада. Где строки R и R + 1 перестают различаться, тексель за этим местом показывает, сдвинулся ли край на строку вверх или вниз (ступенька: край пересекает середину ступеньки, поэтому она входит в подгонку, и прослеживание продолжается) или кончился (угол или узор, не похожий ни на то, ни на другое). В подгонку входят не больше 4 ступенек в каждую сторону.
4. Край как прямая: прямая наименьших квадратов через середины ступенек (x_k, y_k), каждая с весом по расстоянию от пикселя, $w_k = \exp(-x_k^2 / 2\sigma^2)$, где σ = 4 тексела:

   > $\text{minimise}\ \Sigma_k\, w_k\,(y_k - a - b\,x_k)^2 \ \Rightarrow\ b = (\Sigma w\,\Sigma wxy - \Sigma wx\,\Sigma wy) / (\Sigma w\,\Sigma wx^2 - (\Sigma wx)^2),\ \ a = (\Sigma wy - b\,\Sigma wx) / \Sigma w$

   Прямая используется, только если середины лежат от неё в пределах 0.3 текселя по RMS. Одиночная ступенька плавно сходит на нет к ровному участку в пределах дальности прослеживания (форма L, расставленная так, как её расставляет морфологическое сглаживание). Когда первые ступеньки в обе стороны идут в одну сторону (форма U: бугор, верх круглого предмета), прямая изгибается посередине участка, где лежит пиксель. Если ступенек в досягаемости нет, край лежит ровно, посередине между строками.
5. Освещённая доля это профиль треугольного фильтра поперёк этой прямой: прямой край такой мягкости, какую просила игра, без ступенек текселей. Для знакового расстояния s от прямой в текселях и полуширины треугольника H:

   > $\text{lit}(s) = ½ + s/H - s|s| / (2H^2)\ \ \text{при}\ |s| \le H,\ 0\ \text{или}\ 1\ \text{за пределами}$

   Это интеграл нормализованного треугольника (1 − |x|/H)/H до s.

Широкие полутени, начиная с полуширины треугольника 1.25 текселя, берут треугольный фильтр по текселям: их ступеньки и так размыты, а окно 4 × 4 не видит край так далеко. Начиная с половины этой ширины оба способа смешиваются. Узоры, похожие на текстуру (листья, трава: много изменений внутри 4 × 4), тоже берут треугольный фильтр.

Blocker search: насколько мягок каждый край

Тени резкие там, где объект касается земли, и мягчеют с расстоянием. Игра оценивает это как PCSS: усредняет глубины блокеров, найденных вокруг пикселя, d_{avg}, и превращает отношение

> $p = (d_{recv} - d_{avg}) / d_{avg}$

в размер полутени. Поиск игры читает 16 текселей с фиксированными смещениями; когда они по одному пересекают край блокера, размер меняется заметными скачками. Поиск Next Gen (hq_blocker) читает 64 тексела на сетке 4 × 4 выборок Gather по той же площади (внешние ячейки с центрами на 0.63 радиуса поиска, куда доходят стоковые смещения), со стоковым правилом, что каждый тексель считается как min(depth, d_{recv} − 0.001), и возвращает то же отношение. Игра превращает его в полутень точно так же, как раньше; только теперь оно меняется плавно.

Дизеринг швов

На каждой границе каскада тень перескакивает с текселей и ядра одного каскада на тексели и ядро следующего (тексели от 2 до 2.5 раз крупнее, ядро уже в e раз): на земле появляется линия в 15.5 м от камеры. Патч шва заменяет сравнение при выборе каскада. Перед сравнением глубина пикселя z увеличивается максимум на 8 % от себя самой с шумом n, своим для каждого пикселя и кадра:

> $z' = z\,(1 + 0.08\,n), \quad n \in [0, 1)$ (interleaved gradient noise, сдвинутый по времени игры)

Поэтому на последних 0.08 / 1.08 = 7.4 % диапазона каждого каскада (от 14.4 до 15.5 м, от 37.5 до 40.5 м, от 93.1 до 100.5 м) растущая доля пикселей берёт следующий каскад, и TAA игры усредняет оба. Более близкий каскад пиксели не берут никогда.

Ограничения

- Ревекторизация предполагает прямые края. Детали мельче текселя (тонкие ветки, провода) нельзя восстановить из текселей, которые их никогда не видели; там фильтр оставляет треугольник.
- Дизеринг швов опирается на TAA; без TAA полоса видна как точечный переход.
- Resolution стоит времени на проходе теней и памяти: Factor 2 даёт вчетверо больше текселей.

Контактные тени

Карточка Contact shadows. Мелкие солнечные тени, которые трассируются по экрану там, где предметы касаются друг друга или перекрываются: под шиной, где одно бревно лежит на другом, в складках кабины.

Почему карта теней их пропускает

Один тексель теней покрывает от сантиметров до дециметров земли, а смещение (bias), которое спасает от «акне» теней (shadow acne), отодвигает сравнение ещё дальше. Тени мельче этого, например тонкая тёмная линия там, где колесо встречается с грязью, проваливаются между текселями. У буфера глубины самой камеры разрешение в пиксель, поэтому трассировка в сторону солнца в экранном пространстве (screen space) их находит.

Трассировка (screen-space shadows от Bend Studio)

Проход построен на методе экранных теней Bend Studio (из Days Gone; Apache 2.0, изменённая копия входит в DLL). Солнце проецируется в точку на экране. Каждый пиксель лежит на прямой через эту точку, поэтому экран режется на лучи, расходящиеся от неё, и один волновой фронт (wavefront) из 64 потоков проходит 64 соседних пикселя одной прямой вместе, делясь прочитанными глубинами. Для каждого пикселя читается 60 выборок глубины в сторону солнца вдоль его прямой: контактная тень может быть длиной до 60 пикселей. Выборка заслоняет, если лежит перед лучом от пикселя к солнцу; каждая выборка изображает поверхность заданной толщины (ContactThickness, 0.005 в единицах глубины прохода); контраст 4 делает результат резче. Первые 4 выборки считаются жёсткой тенью, последние 8 плавно гаснут, а результат это маска m от 1 (освещено) до 0 (в тени). Проход читает линейную глубину игры как 1/z и пропускает пиксели дальше 50 м (ContactMaxDepth).

В картинку: только солнечный свет

Экранная тень не должна затемнять свет, который не от солнца. Поэтому патч (модуль contact) заставляет каждый из 1269 шейдеров материалов, принимающих солнце, записывать найденную видимость солнца (освещённую долю по карте теней после нижнего порога игры, g_fShadowAtten) в рендер-таргет 6. После прохода освещения DLL рисует полноэкранный проход, который умножает цвет на

> $$k = 1 - (1 - m) \cdot \mathrm{share} \cdot \mathrm{strength} \cdot \mathrm{fade}$$

> $$\mathrm{share} = \mathrm{sun} / (\mathrm{sun} + \mathrm{sky}), \quad \mathrm{sun} = L(C_{\mathrm{sun}}) \cdot m_{\mathrm{pbr}} \cdot \max(0, N \cdot L) \cdot v, \quad \mathrm{sky} = L(\text{три цвета неба, смешанные по } N_y)$$

где v это видимость из рендер-таргета 6, C_{sun} цвет солнца в игре, m_{pbr} его PBR-множитель, L() яркость (luminance), N нормаль для освещения (рендер-таргет 7, где материал её записал, иначе восстановленная по буферу глубины). Пиксель, полностью освещённый солнцем, темнеет примерно до того, что дала бы там тень от карты теней; пиксель, уже стоящий в тени или отвёрнутый от солнца, не трогается. С расстоянием эффект гаснет: fade = 1 − smoothstep(30 m, 50 m, z). Вдали контактная тень меньше пикселя, а земля, видимая под пологим углом, начинает в экранном пространстве затенять саму себя.

Ограничения

- Только экранное пространство: заслоняющий объект за краем кадра или скрытый за чем-то более близким ничего не отбрасывает.
- Толщина одна на всё: очень тонкие предметы (провода) могут отбрасывать за собой слишком длинную тень.
- F4 выключает и включает проход.

Тени от фар

Карточка Headlight shadows, включена по умолчанию. Фары грузовика отбрасывают тени: столб, дерево или камень в луче оставляют землю за собой тёмной.

Как игра освещает фарами

Фара это проецируемый прожектор (spot light). В CB_GLOBAL_SCENE лежат её положение (g_vHeadLightPos), направление, конус, затухание с расстоянием и цвет, а также матрица, которая проецирует мировую позицию в световой рисунок фары (g_tmHeadLightViewProj). Каждый из примерно 5100 пиксельных шейдеров освещения, принимающих свет фар, меряет пиксель от фары, проверяет дальность и конус, гасит свет в пределах от 3 до 6 м от фары и применяет затухание с расстоянием и цвет. Шейдеры объектов (около 2600) к тому же один раз берут выборку из текстуры рисунка g_txHeadLightMap; шейдеры ландшафта (около 2300) считают интенсивность сами и умножают на неё цвет фары. Ни в одной из этих цепочек не спрашивается, стоит ли что-то между фарой и пикселем: карты теней для фар в игре нет.

Проход к фарам

Патч (модуль headshadow) вставляет вспомогательную функцию сразу после выборки рисунка и умножает выборку на то, что функция вернёт, а в шейдерах ландшафта вставляет её прямо перед умножением на цвет фары и умножает на результат интенсивность: v, доля дошедшего света. Для пикселя в точке P и фары в точке L функция идёт от точки в 0.12 м над поверхностью к L и останавливается за 2.5 м до фары (там стоит кузов самого грузовика, да и свет так близко игра всё равно уже погасила). Оба конца проецируются матрицей вида и проекции (view-projection) прошлого кадра (cb1 c10..c13), так как глубина, с которой их сравнивают, взята из того кадра, и трассировка делает один шаг на каждые 6 пикселей своей длины на экране, не меньше 24 и не больше 64, через равные промежутки, так что столб шире промежутка между двумя шагами не перешагивается (более тонкий может быть: см. ограничения). С постоянными 24 шагами длинная трассировка перескакивала через перекладину забора толщиной в несколько пикселей, и её тень выходила гребёнкой полос, тем хуже, чем шире поле зрения (замечено в игре, 2026-10-27). Расстояние шага от камеры интерполируется как 1/w:

> $$w(f) = 1 \,/\, \mathrm{lerp}(1/w_0, 1/w_1, f), \;\; f = (i + j) \,/\, n$$

где j это сдвиг для каждого пикселя (interleaved gradient noise, Jimenez 2014), который усредняет TAA игры. На каждом шаге линейная глубина сцены z в этом пикселе (копия глубины прошлого кадра от SnowRunner Shadows, t121, снятая на проходе AO, который идёт после прохода освещения) сравнивается с w:

> $$d = w - z - (0.04 + 0.006\,w), \;\; b = \mathrm{saturate}(d \,/\, 0.15) \cdot [d < 1.5 + 0.03\,w] \cdot e$$

d > 0 значит, что сцена стоит перед шагом. Если перед ним больше, чем на толщину (1.5 м плюс 3 % расстояния), луч прошёл позади отбрасывающего тень предмета, а не сквозь него. e гасит проверку на последних 3 % экрана у его края. Видимость фары равна 1 − max b по шагам.

То, что заслоняет, ставится в мир: на прямую камеры через шаг, на глубину сцены в этом месте, O = eye + (S − eye) · z / w, где S это собственная позиция шага. Препятствие меньше чем в 0.3 м перед плоскостью фар не считается: ничто позади фары не может стоять между ней и освещённой точкой. Без этого правила собственная кабина грузовика и багажник на крыше, видимые с камеры сверху и

сзади, оказывались в пределах толщины перед дорогой и затемняли луч рядом с грузовиком (замечено в игре, 2026-10-23).

Лучи низко поставленной фары уходят от земли под пологим углом: первый метр они идут в нескольких сантиметрах над ней. Там их заслоняли мелкие комья снега, для каждого сдвига пикселя в другой точке: мелкое зерно по всему освещённому снегу, а на некоторых склонах полосы (замечено в игре, 2026-10-26). Теперь каждый луч начинается на 10 см выше поверхности, строго вверх (HS_LIFT), поэтому комья ниже этого не заслоняют. В тестовой сцене с сугробами по 12 см, неровностью 4 см и без отбрасывающих тень предметов доля луча, считанная заслонённой, равна 0.01 % из-за грузовика и 0.02 % с камеры преследования, против 5.1 % и 5.2 % без подъёма. Второе изменение, опробованное вместе с ним, позволяло там, где шаг казался заслонённым, засчитывать более дальнюю из глубин на ряд пикселей выше и ниже (HS_ROWS, всё ещё в коде); в тестовой сцене оно убрало остальное, но порезало тени тонких перекладин гребёнкой, поэтому выключено.

Не считается и препятствие в пределах 2.5 м от фары (HS_BODY, столько же, где останавливается проход). Проход туда никогда не доходит, так что то, что камера видит там, это собственный перед грузовика (бампер, лебёдка, кенгурятник), найденный только там, где он стоял перед шагом дальше. С низкой точки сбоку от бампера он заслонял каждый луч, проходящий позади него: на дороге выходил тёмный четырёхугольник с жёсткими краями, верхний край вровень с бампером (замечено в игре, 2026-10-24). В виде тестовой сцены сбоку от бампера это правило освещает 14 272 из 175 948 пикселей луча, которые были заслонены, сбоку от грузовика 1 585, и ничего не меняет сзади и с камеры преследования; столбы, отбойник, стена и опора отбрасывают тени как раньше. В игре то же правило убрало и тёмные пятна с жёсткими краями на дороге перед грузовиком, едущим в гору, которые выглядели так, будто тень отбрасывает меш самой дороги (замечено в игре, 2026-10-24).

Первые сборки проецировали шаги камерой текущего кадра. При неподвижной камере это одно и то же; при движущейся (тряска на плохой дороге) каждый шаг встречал то, что было в этом месте экрана в прошлом кадре, и дорога впереди темнела, пока камера не успокаивалась (замечено в игре, 2026-10-23). С камерой прошлого кадра шаг встречает ту глубину, которую там видела та камера; O по-прежнему ставится от глаза текущего кадра, отстающего на движение за кадр, и это сдвигает её на это движение, умноженное на (1 − z / w), то есть на несколько сантиметров.

Проход начинается на поверхности, которую видела камера, в собственной точке пикселя: там, где луч камеры через пиксель встречает глубину. Вставка передаёт функции место пикселя на экране (собственный SV_Position шейдера: 3 822 из 5 123 шейдеров уже читают его, 1 300 получают его объявленным, так, как его называет их сигнатура), и функция читает глубину в этом пикселе. Глубина взята из прошлого кадра, точна, пока камера стоит; пока она движется, точка дважды перепроецируется в прошлый кадр и сдвигается вдоль луча на то, что говорит глубина там. Точка, которую прошлый кадр не видел (глубина остаётся дальше чем на 15 см плюс 2 % расстояния), на этот кадр сохраняет свет самой игры.

Позиция, которую получает шейдер, для этого не используется. На ландшафте она не лежит на поверхности, которую видела камера: на освещённом снегу немного в стороне в каждом пикселе, на снежном валу перед грузовиком на метры в любую сторону. Тогда каждый проход начинался внутри изображения глубины или чуть над ним и встречал собственные шаги, что рисовало зерно и полосы по снегу, сильнее без TAA, а на валу большое тёмное пятно с прямыми краями (замечено в игре, 2026-10-27). В первой версии функции их не было только потому, что она не затеняла ландшафт: шейдеры ландшафта, которые освещают фарами без рисунка, добавились днём позже. Перенос позиции шейдера на глубину вдоль её собственной линии взгляда (первая попытка в 1.1.1) оставил полосы, так как эта позиция не проецируется в свой пиксель. При старте от собственного луча пикселя полосы и зерно в том же месте пропали, с TAA и без. Единственный шейдер, в сигнатуре которого нет экранной позиции, по-прежнему переносит свою позицию на глубину этим способом.

Две фары грузовика стоят примерно в 1.4 м друг от друга, а игра передаёт шейдерам одну позицию между ними. Функция идёт к точке в 0.7 м в каждую сторону, горизонтально и поперёк направления фары, и усредняет оба результата, так что пиксель, до которого достаёт только одна фара, получает половину света: край тени выходит полуосвещённой полосой, тем шире, чем ближе к грузовику стоит отбрасывающий тень предмет. Это не больше 128 чтений глубины на пиксель в луче и ни одного вне его, так как вставка стоит внутри собственной проверки игры на дальность и конус.

Какие шейдеры

Функции нужна мировая позиция пикселя в том виде, в каком шейдер мерил её от фары: вход из вершинного шейдера или временный регистр, причём только там, где ни одна инструкция не перезаписывает его до вызова функции. 5123 из 5143 шейдеров её дают; остальные 20 (частицы, у которых позиция приходит без третьей компоненты или не приходит вовсе) сохраняют свет игры. В каждом пропатченном шейдере объявляются константы камеры вплоть до view-projection прошлого кадра (c13) и объявляется t121. Шейдеры травы и кустов (267) принимают свет фар, но не объявляют констант камеры вовсе; патч объявляет их сам (b1, который каждый использующий его пиксельный шейдер игры привязывает к CB_GLOBAL_CAMERA). До 2026-10-23 их пропускали, и столб или отбойник на обочине ничего не отбрасывали, а дорожный конус на асфальте отбрасывал (замечено в игре): их тени падают на траву. Модуль патчится последним, поверх того, что остальные модули сделали из каждого шейдера. Все 5123 проходят входную проверку сборки и создаются в Direct3D 11, на WARP и драйвером GPU (проверено офлайн, 2026-10-23). Первая сборка доставала только до объектов: в игре тень столба падала на опору за ним, но не на землю, и так выяснился собственный способ освещения ландшафта.

Ограничения

- Только экранное пространство: предмет за краем кадра или скрытый за чем-то, что ближе к камере, ничего не отбрасывает, поэтому тень может меняться при повороте камеры.
- Ничто в пределах 2.5 м от фар не отбрасывает тень: собственные бампер, лебёдка и кенгурятник грузовика не отбрасывают, как и столб так близко.
- Тонкие предметы всё ещё можно пропустить: там, где трассировка на экране длиннее 384 пикселей, его 64 шага стоят дальше друг от друга, чем ширина тонкого столба. Следующий шаг, если игра всё ещё покажет такие пропуски теперь, когда тени ложатся и на траву (см. «Какие шейдеры»), это трассировка по пирамиде ближайших глубин (t125, от прохода отражений), который ничего не может перешагнуть.
- Дальний край приподнятого наложения (overlay) может затенить короткую полосу земли за ним, как это сделал бы шаг на такой высоте (см. начало прохода).
- Густая трава рисуется по нескольку раз, и каждый слой в луче теперь идёт к фарам: там функция стоит больше, чем указано ниже.
- В игре есть слоты на четыре фары; затеняется только первая.
- Без SnowRunner Shadows или через 250 мс после того, как его копии прекращаются (меню), t121 не привязан, и свет остаётся светом игры.
- В игре работу видели один раз (тень столба на опоре за ним, в сравнении по F8); ещё не настроено.

Проверка без игры

Офлайн-сцена (test\headshadow_scene) трассирует лучами на каждый пиксель синтетическую ночную сцену: земля, кузов грузовика у фар, столбы, опора, бетонный блок, низкий блок в 1.6 м перед бампером, стена, отбойник на столбах вдоль левой стороны дороги и тонкий столбик-указатель; OVERLAY_H поднимает глубину участка дороги над его затеняемыми пикселями, как это делает

наложение. Она записывает линейную глубину сцены так, как её хранит t121 в DLL, заполняет константы камеры и фар, как это делает игра, и затеняет сцену самой функцией: сбоку от грузовика, сзади (камера преследования), сверху, низко перед ним сбоку от бампера и высоко сзади подальше (последние два как на снимках тёмного пятна из игры). Каждую настройку там можно переопределить. Видно в ней: тени уходят от фар, вблизи двоятся; с камерой преследования собственная кабина грузовика ничего не заслоняет; при 1.5 м вместо 2.5 м низкий блок тоже отбрасывает тень, а грузовик по-прежнему ничего не заслоняет, что у настоящего кенгурятника или лебёдки может быть не так.

Её measure.ps1 меряет одну функцию против того же прохода без неё, одна выборка на пиксель, как в игре, лучший из 60 вызовов отрисовки на RTX 4080: при 3840 × 2160 0.19 мс, когда в луче 5 % экрана (камера преследования), от 0.52 до 0.84 мс при доле от 15 до 23 %, 1.2 мс с камерой низко за фарами и 39 %. Не больше чем с 24 шагами, как в 1.1.0, те же виды занимают 0.05, от 0.26 до 0.35 и 0.54 мс: более длинные трассировки стоят примерно вдвое дороже. В игре шейдеры освещения выполняются один раз на каждую нарисованную поверхность, поэтому места, нарисованные по нескольку раз (трава, листва), платят больше. Измерено в игре при 3840 × 2160 на RTX 4080 (время GPU прохода освещения, грузовик стоит, луч заполняет вид впереди): 4.6 мс без теней, 9.0 мс с ними, то есть около 4.4 мс; при повороте камеры частота кадров остаётся той же, что и на стоянке, так как ничего не перерисовывается.

Фоновое затенение: GTAO и широкий радиус

Карточки GTAO (строка: half или full size) и Wide occlusion. Тень в углах, под грузовиками и в колёсных арках, куда доходит меньше неба.

Что измеряет затенение

Фоновый свет приходит со всего неба. Точка в углу видит его меньше. Измеряемая величина это видимость V: доля полусферы над поверхностью, которая открыта, взвешенная косинусом к нормали (свет строго сверху весит больше всего):

$$V = (1/\pi) \int_\Omega \text{visible}(\omega)\ (N \cdot \omega)\ d\omega$$

Собственный проход игры оценивает её по 8 выборкам глубины на постоянном ядре, размер которого составляет постоянную долю экрана при любом расстоянии: ящик выглядит одинаково затенённым, будь он в 2 м или в 200 м.

GTAO: интегрирование по горизонтам

Ground truth ambient occlusion (Jimenez et al. 2016, «Practical Real-Time Strategies for Accurate Indirect Occlusion»; интегрирование сделано по XeGTAO от Intel, MIT) делит полусферу на срезы: плоскости через направление взгляда под экранными углами φ. В каждом срезе открытое небо это дуга между двумя горизонтами, h_1 с одной стороны и h_2 с другой: самый крутой угол на любую поверхность в пределах радиуса. Интеграл по этой дуге берётся в замкнутом виде. Пусть n это угол нормали, спроецированной в срез, а $|n_p|$ длина этой проекции:

$$a(h) = ¼\ (-\cos(2h - n) + \cos n + 2h \sin n)$$

$$V \approx (1/S)\ \Sigma_{\text{slices}}\ |n_p|\ (a(h_1) + a(h_2)),\quad AO = V^{1.6}$$



Один срез: открытая дуга между двумя горизонтами это доля неба, которую точка видит в этом направлении.

Числа: 4 среза на пиксель, 8 выборок глубины на каждую сторону среза (64 чтения глубины), радиус 1.5 м, ограниченный 12 % глубины пикселя, чтобы ядро вблизи камеры оставалось ограниченным на экране. Нормаль восстанавливается по буферу глубины. Углы срезов и положения выборок поворачиваются для каждого пикселя шумом interleaved gradient noise; шум усредняет собственное размытие 3 × 3 игры в её проходе наложения. Эффект гаснет между 150 м и 300 м, где берёт своё туман. Поле зрения каждый кадр берётся из камеры игры. Проход заменяет два шейдера SSAO игры и сохраняет их входы и выходы: x = видимость, yzw = глубина центра для размытия.

Половинный размер

Со строкой GTAO в значении half size (и AOHalf = 1 в SnowRunner Shadows) проход идёт по четверти пикселей, каждый пиксель половинного размера привязывается к глубине одного полноразмерного пикселя, который он покрывает, а DLL увеличивает результат с весами по глубине, так что затенение не перетекает через края объектов. Результат ещё и копируется в собственный AO-таргет игры: при половинном размере проход туда не рисовал, и читатель, до которого не дошла подмена в DLL, рисовал последний кадр, сделанный в полном размере, бледную картинку старого вида, которая оставалась на экране, куда бы ни повернулась камера (сообщения игроков, 1.1.1). Полный размер стоит по умолчанию с 1.1.2.

Широкое затенение

Крупные формы (кузов грузовика, крыша, крона дерева) затеняют землю дальше чем на 1.5 м. Широкий радиус продолжает каждый поиск за ближний радиус до 6 м, ещё 4 выборками на сторону, с ограничением в 50 % глубины вблизи камеры. Дальний заслоняющий объект считается не полностью: на ближнем радиусе он поднимает горизонт наполовину (сила 0.5), к 6 м сходя на нет, так что тень под грузовиком мягкая, а чёткое контактное затенение у земли остаётся каким было. Для экономии времени каждый квад 2 × 2 пикселя делит дальний поиск: каждый пиксель ищет далеко в одном из четырёх направлений, и четыре результата делятся между всеми. Это четверть дальних чтений: 0.9 мс вместо 3.3 мс при 3840 × 2160.

Ограничения

- Экранное пространство: то, что за кадром или скрыто за чем-то, затенения не даёт.
- Тонкие предметы перед стеной дают на стене тёмный ореол; взвешивание по глубине и затухание держат его маленьким.
- F11 сравнивает GTAO с SSAO игры, F5 переключает между половинным и полным размером.

Переотражённый свет

Карточка Bounce light (выключена по умолчанию, экспериментальная). Освещённая солнцем земля и краска подсвечивают своим цветом то, что стоит рядом.

Идея: горизонты уже знают, кто закрывает небо

Поиск GTAO находит в каждом направлении, какие поверхности закрывают небо. Поверхность, которая закрывает часть неба, вместо него посылает свой свет. Каждая выборка, поднимающая горизонт с h до h', закрывает ещё одну дугу неба, и эта дуга теперь видит найденную выборкой поверхность. Её доля это та же взвешенная косинусом мера, которую интегрирует GTAO, a(h') − a(h). Поэтому переотражённый свет равен

> $$B = s \cdot (1/S) \, \Sigma_{\text{slices, sides}} \, \Sigma_{\text{taps that raise the horizon}} \, |n_p| \, (a(h') - a(h)) \cdot \min(C_{tap}, 64)$$

где C_{tap} это цвет выборки в освещённом изображении этого кадра (копия в DLL, t120), обрезанный на 64, чтобы отбросить солнечные блики, а s = 4 это сила. Пиксель, окружённый освещённым снегом, собирает свет снега; пиксель рядом с красным грузовиком собирает красный. (Вывод наш собственный, из интеграла по дуге GTAO; идея та же, что у непрямого освещения по горизонтам.)

Как удержать обратную связь устойчивой

Переотражённый свет становится частью освещённого изображения, из которого собирает следующий кадр: свет может переотражаться снова и снова. Два ограничения держат петлю в рамках. Сохранённое значение никогда не превышает 0.9 от самой яркой поверхности, с которой оно собрано. А сторона материала никогда не добавляет больше заданного кратного от самого яркого из своих фоновых цветов.

В материалы

Игра освещает тень тремя фоновыми цветами: сверху, с горизонта и снизу (cPosY, cMidY, cNegY), смешанными по нормали и умноженными на цвет поверхности. Патч (gi_ambient) добавляет переотражённый свет ко всем трём, так что собственный код игры доносит его дальше с настоящим альбедо. Поскольку переотражение измерено в прошлом кадре, каждый пиксель ищет его там, где он был тогда: проецируя камерой прошлого кадра, если мир стоял на месте, или в той же позиции экрана, если он двигался вместе с камерой (грузовик игрока). Какой случай подходит, решает глубина прошлого кадра; где не совпадает ни один (только что открывшееся место), переотражённого света в этом кадре нет. Проход AO хранит переотражение, делённое на затемнение, которое пикселю даст проход наложения AO, так что после наложения переотражение сохраняет свою силу.

Ограничения: он экспериментальный, потому что местами (яркий снег рядом с тёмным деревом) выходит слишком сильным, и, как любой эффект в экранном пространстве, знает только то, что на экране. F10 выключает и включает его.

Фоновое освещение неба и заполняющий свет

Карточки Sky ambient и Fill light. Цвет и количество света, который заполняет тень.

Sky ambient

Каждое дневное состояние задаёт три фоновых цвета как один тёплый серый трёх разных яркостей, поэтому у тени нет цветовых оттенков: под синим небом она должна уходить в синеву, на закате в тепло. Патч (sky_ambient, вставлен в начало каждого шейдера материала, который читает эти цвета) сохраняет яркость каждого цвета и берёт оттенок неба в его направлении из кубической карты отражений, которую шейдер и так привязывает, на её самом шероховатом мип-уровне (близко к облучённости неба): строго вверх для cPosY, строго вниз для cNegY, среднее четырёх горизонтальных направлений для cMidY.

> $$c' = \mathrm{lerp}(c, L(c) \cdot S / L(S), 0.6)$$

где c это заданный автором цвет, S цвет кубмапы в этом направлении, а L() яркость. Собственное смешивание игры по нормали затем работает уже с новыми цветами.

Fill light

Дневные состояния дают тени много света: в солнечный день в Зурдании свет сверху равен × 3.9 против × 5.15 у солнца. Если сравнить с нейросетевыми рендерами тех же сцен, то затемняют они именно этот заполняющий свет. Карточка масштабирует три фоновых множителя каждого дневного, вечернего и утреннего состояния в initial.pak (199 состояний; ночные, гаражные и состояния меню не трогаются) одним коэффициентом f. Поскольку заполняющий свет даже на освещённой солнцем земле составляет около половины всего света, его уменьшение затемняет всю картинку, и адаптация глаза в игре подняла бы всё обратно. Поэтому экспозиция каждого состояния тоже масштабируется:

> $$\mathrm{ambient} \times f, \quad \mathrm{HdrExposureScale} \times f^{0.45} \quad (f = 0.7 \to 0.852;\ 0.85 \to 0.929)$$

Так освещённые солнцем места остаются примерно там же, где были, а тень становится темнее. Правку в initial.pak делает движок (см. главу о движке).

Object reflections: проход отражений, reflection pass (SSSR)

Карточка: Object reflections, вместе с SnowRunner Shadows. Краска, стекло и хром отражают окружающую сцену. Название SSSR взято из FidelityFX Stochastic Screen Space Reflections от AMD: проход использует их обход луча.

Что делает игра без него

Глянцевый материал в SnowRunner отражает только одно: кубическую карту отражений (кубмапу), то есть картинку неба на шести квадратных гранях, одну на всю карту. Материал читает её в направлении отражённого луча взгляда и размывает по шероховатости через цепочку мип-уровней кубмапы: мип-уровень = √roughness × 8 + 0.5. Поэтому на капоте грузовика видно нарисованное небо, даже если рядом стоит стена. Отражения в экранном пространстве (screen space reflections) добавляют то, что видит камера.

Принцип

Для каждого пикселя известны его положение P, направление к глазу V и нормаль поверхности N. Идеальное зеркало продолжает луч взгляда в направлении

> $$R = 2\,(N \cdot V)\,N - V$$

Если этот луч попадает во что-то, что видит и камера, цвет точки попадания и есть отражение. Сцена, которую видит камера, известна в виде двух картинок: освещённого цвета и глубины каждого пикселя. Значит, поиск сводится к шагам вдоль R по экрану: глубина луча сравнивается с глубиной, записанной в каждом пройденном пикселе, пока луч не уйдёт за поверхность.

Откуда берутся нормаль и шероховатость

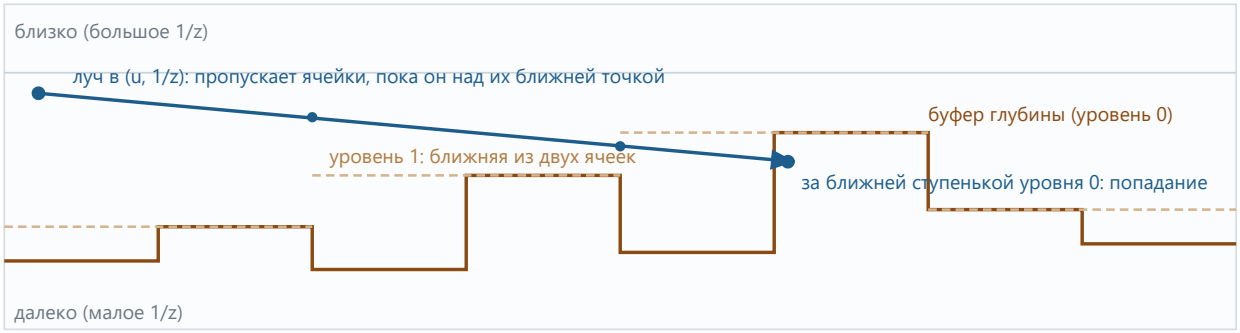
Игра рисует мир за один проход, в котором каждый материал сразу считает итоговый цвет; отдельного буфера нормалей у неё нет. Модуль sssr из Next Gen встраивает небольшой фрагмент записи во все 3229 шейдеров материалов техники и объектов, в место их выборки из кубмапы: он пишет нормаль после шейдинга (в мировом пространстве, × 0.5 + 0.5) и перцептивную шероховатость r в дополнительный рендер-таргет RT7 (RGBA8). DLL подключает его рядом с проходом освещения и после него очищает значением «вверх, полностью шероховато». Пиксель с r ≥ 0.999 луч не трассирует.

Прямые на экране, и почему 1/z

Луч из точки O = P + N (0.002 z + 0.01) (приподнят над поверхностью на 1 см плюс 0.2 % её расстояния z, чтобы не попасть в саму себя) проецируется в экранные координаты на обоих концах. Прямая в мире остаётся прямой на экране, но глубина z меняется вдоль неё нелинейно, а обратная величина 1/z линейно. Поэтому проход везде хранит 1/z: луч задан как o + t d в (u, v, 1/z), и пирамида глубины тоже хранит 1/z. Чем больше 1/z, тем ближе.

Пирамида глубины (Hi-Z)

Уровень 0 хранит 1/z каждого пикселя (0 для неба, z ≥ 3000 м, чтобы ни один луч в него не попал). Каждый следующий, более грубый уровень вдвое меньше и хранит для каждой ячейки *наибольшее* 1/z из четырёх (на нечётном краю из шести или девяти) ячеек под ней: ближайшую поверхность во всей ячейке. Луч, который ближе этого значения, ни во что в этой ячейке попасть не может.



Обход пирамиды по одной оси экрана. Над ячейками, чья ближайшая поверхность лежит за лучом, луч идёт длинными шагами грубого уровня; там, где он пересёк бы поверхность, он спускается на уровень ниже, вплоть до отдельных пикселей.

Обход

Луч стартует на уровне 0 и сдвигается к дальнему краю пикселя, в котором начинается. Затем, не более 80 раз (SSRSteps):

1. Прочитать ближайшую поверхность ячейки s на текущем уровне.
2. Найти, где луч выходит из ячейки: ближайшее из его пересечений с двумя граничными линиями ячейки и, пока луч удаляется от камеры, с плоскостью 1/z = s. Это t_{min}.
3. Если луч перед ближайшей поверхностью ячейки (s < его собственного 1/z), сдвинуть его в t_{min}. Если он вышел через край, а не через плоскость поверхности, подняться на уровень выше: следующие ячейки крупнее. Иначе спуститься на уровень ниже.
4. Остановиться, когда уровень опустится ниже 0: луч стоит за поверхностью с точностью до пикселя. Или когда он уйдёт за экран.

Над открытой землёй обход проходит много пикселей за несколько шагов; рядом с геометрией он замедляется до шагов в пиксель. Это метод Hi-Z трассировки Uludag в том виде, в каком AMD написала его для FidelityFX SSSR; проход адаптирует её функции FFX_SSSR_InitialAdvanceRay, FFX_SSSR_AdvanceRay и FFX_SSSR_HierarchicalRaymarch (лицензия MIT, уведомление в DLL).

Настоящее ли это попадание?

Точка за буфером глубины не обязательно лежит на поверхности: буфер хранит только переднюю сторону предметов. В пикселе, где закончился обход, проход проверяет:

Проверка	Правило
на экране и сдвинулся	попадание внутри экрана и дальше одного пикселя от точки старта луча
не небо	записанная глубина z_s меньше 3000 м
лицевая грань	нормаль поверхности попадания N_s смотрит на луч: $N_s \cdot R \le 0$
толщина	насколько глубоко за поверхностью стоит луч, $b = z_{ray} - z_s$, в сравнении с принятой толщиной предметов $T = 0.3 \text{ м} \times (1 + 0.02\, z_s)$

По толщине попаданию даётся уверенность c от 0 до 1:

> $$c = (1 - \text{smoothstep}(0, T, b))^2$$

Луч, ушедший за переднюю сторону дальше чем на T, прошёл под объектом: под кабиной, мимо груза в кузове. Если оставить такие пиксели пустыми, материал вернётся к нарисованному небу, и рядом с настоящими отражениями появятся светлые пятна. Поэтому в пределах 2.5 м (плюс 5 % глубины, SSRUnder) за поверхностью луч берёт цвет этого объекта, затемнённый к половине (SSRUnderShade 0.5) на протяжении полуметра после T. Это заменяет днище, которого камера не видит. Полностью это действует на поверхностях до 25 м и сходит на нет к 50 м (SSRUnderNear, SSRUnderFar). Это не применяется за «точкой», маленьким ярким предметом, вокруг которого ничего нет, например светлячком или далёким фонарём: хотя бы с трёх сторон из четырёх, на расстоянии 2 % высоты экрана, глубина другая, а пиксель втрое ярче самого яркого соседа.

Затем уверенность гасится у края экрана (на 5 % его высоты), с ростом шероховатости и для лучей, идущих обратно к камере, цели которых нет ни на одной картинке:

> $$c \times= \mathrm{saturate}((r_{max} - r) \,/\, (0.2\, r_{max})) \times \mathrm{saturate}(2\,(R \cdot -V + 1)),\ \ r_{max} = 0.85$$

Шероховатость: конус

Шероховатая поверхность отражает размытую картинку. По умолчанию каждый луч идёт в зеркальном направлении, а шероховатость только размывает цвет в точке попадания, беря его из цепочки мип-уровней входного изображения. Размытие растёт с расстоянием d (в пикселях), которое луч прошёл по экрану, и с $\alpha = r^2$:

> $$\mathrm{mip} = \log_2(1 + d \cdot \alpha \cdot 0.5)\ \ \ (\mathrm{SSRCone} = 0.5)$$

По желанию (SSRLobe = 1) лучи следуют самому распределению GGX: каждый кадр выбирается новая нормаль микрограни H из распределения нормалей, видимых из V (Heitz 2018), и луч отражается относительно H, а не N. Два равномерных числа u_1, u_2 (interleaved gradient noise, Jimenez, со сдвигом каждый кадр) превращаются в точку на диске, растянутом на α и поднятом на полусферу. Используется только ядро лепестка (u_1 умножено на 0.85), чтобы редкие далёкие лучи не давали искр. Это и есть «stochastic» в SSSR; по умолчанию он выключен, потому что зеркальное направление с размытием по конусу ведёт себя устойчивее при движении в SnowRunner.

Меньше лучей: один на блок 2 × 2

При SSRHalf = 1 (по умолчанию) трассировка идёт по сетке вдвое меньшей ширины и высоты. Каждая ячейка трассирует один пиксель своего блока 2 × 2, и выбранный пиксель меняется за четыре кадра в порядке (0,0), (1,1), (1,0), (0,1), так что каждый пиксель получает свой луч раз в четыре кадра. Это вчетверо снижает стоимость трассировки.

Сборка: от лучей к картинке

Сборка (resolve) идёт в полном разрешении. Каждый отражающий пиксель собирает результаты 3 × 3 соседних ячеек трассировки, ведь луч соседа заменяет параллельный луч из близкой точки. Каждый взвешивается по расстоянию, глубине и нормали, чтобы ничего не переходило через границу:

> $$w = \exp(-|\Delta p|^2 \,/\, \sigma^2) \cdot \mathrm{saturate}(1 - |\Delta z| \,/\, (0.03\, z + 0.02)) \cdot (N_s \cdot N)^{16},\ \ \sigma^2 = 2\, s^2\, (1 + 8\, r^2)$$

где s это шаг сетки (2 при SSRHalf). Из взвешенных выборок получаются среднее μ и разброс σ (стандартное отклонение) по каждому каналу.

Затем временное накопление (temporal). Пиксель ищется там, где он был в прошлом кадре: по репроекции камеры или, на едущем грузовике, по движению объекта из собственного буфера скоростей игры. Прошлый результат там используется, только если совпадает глубина прошлого кадра: в пределах 2 % + 5 см для неподвижного, 10 % + 30 см вдоль движущегося объекта. Затем история подтягивается к среднему этого кадра, пока не окажется в пределах разброса выборок (variance clipping, Salvi 2016). Все каналы сдвигаются на один множитель, так что результат остаётся смесью реально присутствующих цветов:

$$e = \max(1.5\,\sigma,\ 0.01 + 0.02\,|\mu|),\quad m = \max \text{ over channels } |h - \mu| \,/\, e,\quad \text{if } m > 1\text{: } h = \mu + (h - \mu) \,/\, m$$

$$\text{result} = \text{lerp}(\mu, h, k),\quad k = \text{lerp}(0.6,\ 0.9,\ \text{saturate}(r \,/\, 0.3))$$

Зеркала сохраняют мало истории (k = 0.6); шероховатая краска, у которой лучи шумнее, сохраняет больше (до SSRTemporal = 0.9). Где в этом кадре рядом с пикселем не упал ни один луч, его история гаснет с множителем 0.9 за кадр. Выход t123 хранит яркость × уверенность и саму уверенность (RGBA16F).

Обратно в материал

Проход отработал в предыдущем кадре, поэтому шейдер материала репроецирует свой пиксель в тот кадр с видом и проекцией прошлого кадра и проверяет там глубину прошлого кадра (3 % + 5 см; 15 % для пикселей, где прошёл движущийся объект). Затем он смешивает, где a это уверенность:

$$\text{colour} = \text{reflection} + \text{cubemap} \times (1 - a)$$

с ослаблением по глянцу: полностью до мип-уровня кубмапы 2 (очень глянцевое), только кубмапа начиная с мип-уровня 7 (шероховатость около 0.67). Отражённый свет ярче учетверённого света кубмапы сжимается, чтобы фонарь в отражении не пересвечивался. Стекло это особый случай: проход видит кабину за окном, а не само окно. Там, где глубина, которую видел проход, явно лежит за фрагментом (1 % + 2 см + 1.5 пикселя собственного наклона глубины фрагмента), материал вместо этого сам трассирует короткий луч рэймарчингом (ray marching), как пообъектный метод из прежней сборки Object reflections.

Стоимость, ограничения, артефакты

- Стоимость: сборка занимает 0.21 мс при 3840 × 2160 (замер; её ограничивают чтения и записи в полном разрешении). Трассировка это проход в четверть размера, до 80 шагов на луч. Время каждого прохода можно записать в лог с SSRTiming = 1.
- Отразить можно только то, что есть на экране. Того, что вне картинки, за камерой или скрыто за другими предметами, нет в буфере глубины; там отражение переходит в кубмапу, заметнее всего у краёв экрана.
- Толщина это догадка: тонкие столбы могут пропасть; луч может пройти под тонким объектом и захватить землю за ним.
- Задержка в один кадр: при быстрых поворотах камеры видно, как отражение догоняет за кадр, а у только что открывшихся пикселей нет истории.
- F6 в игре выключает и включает проход; когда он выключен, материал сам трассирует луч рэймарчингом (или оставляет кубмапу).

Вода: отражения, эффект Френеля, цвет глубины, свечение гребней

Карточки: Water reflections (строка Sun glitter in shade), River depth colour, Sun glow through waves. Игра рисует воду примерно 200 шейдерами: 20 для рек и грязи, 176 для озёр и морей (вода «domain»), из них 88 с планарным изображением отражения. Next Gen меняет то, что они отражают, то, как смешиваются отражение и собственный цвет воды, и то, как окрашивается глубокая вода.

Отражения в экранном пространстве на реках и простых озёрах

Шейдеры рек и грязи, а также озёра без планарного изображения отражают только кубмапу неба. Патч заменяет их единственную выборку из кубмапы рэймарчингом (ray marching) (ssr.hlsl, по McGuire и Mara, «Efficient GPU Screen-Space Ray Tracing», JCGT 2014, упрощённо для воды, у которой лучи уходят от поверхности вверх). В отличие от прохода для объектов, рэймарчинг идёт в пространстве вида с шагами, растущими в геометрической прогрессии, что хорошо подходит для длинных отражений над ровной водой:

> $\text{step}_k = 0.05\ \text{м} \times 1.25^k,\ \ k = 0 \ldots 35\ \ (\text{последний шаг } 0.05 \times 1.25^{36} \approx 150\ \text{м})$

Первый шаг маленький, чтобы найти колесо, стоящее в луже. На каждом шаге точка луча проецируется, и её глубина сравнивается с линейной глубиной сцены (g_txZ). Когда луч уходит за поверхность, 5 делений пополам между последней точкой перед ней и первой за ней находят пересечение. Попадание засчитывается, если луч не дальше 0.25 м за поверхностью (глубже значит, что он прошёл под объектом); смещение равно 0.02 м плюс доля расстояния. Цвет берётся из непрозрачной сцены, которая у игры уже есть (g_txBBOpaque). Попадания, которые ярче отражённого неба больше чем в 3 раза, сжимаются, чтобы фонари не пересвечивались; отражение гаснет на внешних 8 % экрана. Где луч уходит за экран или ничего не находит, выборка неба остаётся ровно такой, как была.

Планарное изображение озёр

88 шейдеров озёр, морей и луж отражают изображение, которое игра рендерит для одной плоскости воды в половину размера кадра, со своим проходом в экранном пространстве (SpinTires/WaterCompute, «Reflections» и «ReflectionsWithTAA»). Для каждого текселя воды этот проход идёт вверх по экрану по одному текселю за шаг, находит первое, что выступает из воды, и берёт цвет вдвое выше. В его правиле попадания есть изъян: оно требует, чтобы найденное поднималось хотя бы на minRise = 0.01 + 0.04 s метра *на тексель* (s растёт с глубиной). Вертикальный предмет в 20 м поднимается на 3.0 см на тексель при 1080 строках, но лишь на 1.5 см при 2160 строках, поэтому при 3840 × 2160 корпус, шина или пирс в воде находятся только дальше примерно 75 м, и озеро показывает небо там, где должен быть грузовик. Патч (patch_refl_hits.js) делает правило независимым от разрешения:

> $\text{minRise} = 0.5 \times (\text{высота, которую покрывает один тексель на глубине выборки} - \text{tol})$

Поверх этого в месте планарной выборки работает тот же рэймарчинг, что и на реках (ssr_planar.hlsl). Где он попадает, он заменяет планарное изображение (в котором нет грузовиков); где ничего не находит, планарная выборка остаётся. Луч, который закончился заметно за поверхностью,

планарному изображению не передаётся: под объектом он берёт цвет этого объекта, затемнённый, как делает проход для объектов.

Как смешиваются отражение и цвет воды

Свет на поверхности воды делится: доля F отражается, остальное уходит в воду. F следует приближению Шлика (Schlick) для уравнений Френеля, с $F_0 = 0.02$ для воды:

> $$F = F_0 + (1 - F_0)(1 - \cos\theta)^5$$

Шейдеры рек в игре заканчиваются на out = reflection · F + colour, то есть отражение добавляется поверх полного собственного цвета воды. Это работает, пока отражение это яркое небо; с настоящими отражениями тёмных берегов вода становится молочной. Патч (blend.hlsl) заставляет цвет уступать место:

> $$\text{out} = \text{colour} \cdot (1 - F) + \text{reflection} \cdot F$$

кроме солнечного блика, который крупные варианты шейдера добавляют в собственный цвет и который сам уже отражённый светом: его доля, highlight · (1 − T), остаётся как есть.

River depth colour: цветное поглощение

Игра растворяет дно реки под водой в собственном цвете воды одним серым множителем, T = exp(−k · path), так что глубокая вода сереет одинаково быстро во всех каналах. Настоящая вода поглощает одни длины волн быстрее других (закон Бугера-Ламберта-Бера, Beer-Lambert): чистая вода сначала теряет красный и уходит в бирюзу, мутная сначала теряет синий и буреет. Патч (absorb.hlsl) даёт каждому каналу свой показатель, заданный собственным цветом воды c (её освещённый цвет с вычтенным оттенком неба):

> $$\text{rel}_c = c \,/\, \max(c), \;\; a_c = 1 - \ln(\max(\text{rel}_c, 0.02)), \;\; w_c = a_c \,/\, L(a)$$

> $$\tau = -\ln T, \;\; \tau' = \tau^2 \,/\, (\tau + 0.7), \;\; T_c = \exp(-w_c\, \tau')$$

Самый яркий канал получает a = 1, более тёмные больше; веса в среднем по яркости L равны 1, так что общая глубина, заданная художниками, сохраняется и меняется только её оттенок. Серый цвет воды даёт исходное затухание. Вторая строка делает мелководье прозрачнее: в тонком слое воды (малое τ) дно видно гораздо дальше, глубокая вода гаснет как раньше.

Sun glow through waves

Если смотреть на низкое солнце через реку, тонкие верхушки волн светятся: свет входит в заднюю сторону волны и выходит через гребень. Это слагаемое «fast translucency» из работы Barre-Brisebois и Bouchard (GDC 2011): направление на солнце, искривлённое нормалью волны, в сравнении с направлением взгляда:

> $$\text{glow} = 0.15 \cdot C_{\text{sun}} \cdot S \cdot \text{hue} \cdot \text{crest} \cdot \text{saturate}(V \cdot -(L + 0.5\, N))^6$$

где S это видимость солнца в пикселе по карте теней (читается так же, как это делают материалы игры; её получают 12 шейдеров рек, которые читают карту теней), hue это собственный цвет воды, а

crest это вес, растущий по мере того, как грань отклоняется от горизонтали (наклонная грань это тонкий гребень; множитель 4). Оно добавляется вне отражения, как свет, прошедший сквозь воду.

Sun glitter in shade (строка, выключена по умолчанию)

Слагаемое блика на воде в игре частично не учитывает тень от солнца: вода в тени всё равно немного блестит. Строка умножает степенное слагаемое блика в 64 шейдерах озёр и 8 шейдерах рек, рисуемых с картой теней, на

> $$g = \mathrm{saturate}(1 - (1 - S) \,/\, \max(1 - f, 0.001))$$

где S это собственная проверка тени шейдера, а f это нижний предел тени в игре (g_fShadowAtten): там, где проверка говорит, что солнце закрыто, блик гаснет. Там, где это пробовали, проверка игры считала воду освещённой, так что ничего не изменилось; поэтому строка остаётся выключенной.

Ограничения

- Экранное пространство: то, что за экраном, не отражается; отражение гаснет у краёв экрана.
- Марш идёт шагами: очень тонкие предметы (трос) на расстоянии можно перешагнуть.
- Свечению нужна карта теней; 8 из 20 шейдеров рек её не читают и свечения не получают.

Отражения на мокрой земле и ограничение бликов фар

Карточки: Wet ground reflections, Headlight glare cap.

Отражения в грязи и лужах

Ландшафт освещает свои отражения только кубмапой неба (с учётом местного фонового освещения), так что лужа рядом с грузовиком отражает небо и больше ничего. Ландшафт рисуется раньше, чем у игры появляется изображение сцены, поэтому патч (puddle_ssr, в 1208 шейдерах мокрой земли) читает изображение предыдущего кадра, которое DLL копирует на проходе AO и подключает в t120 (цвет) и t121 (глубина). Луч идёт в мировом пространстве и проецируется камерой предыдущего кадра, так что глубина и цвет, которые он встречает, соответствуют друг другу; у едущего грузовика отражение отстаёт на кадр. Марш делают только глянцевые пиксели: мип-уровень кубмапы, √roughness × 8 + 0.5, должен быть ниже предела. Шероховатая грязь берёт цвет из цепочки мип-уровней входного изображения, размытый так, как была бы размыта кубмапа. Без DLL (или с F9) каждый пиксель сохраняет выборку из кубмапы.

Ограничение бликов фар

Ландшафт освещает каждую поверхность бликом GGX от каждой фары. Его слагаемое распределения D игра ограничивает значением 32: min(D, 32). На гладкой, покрытой рябью поверхности дорожной лужи это ограничение достигается на длинном отрезке перед грузовиком, и фары рисуют на воде длинную яркую полосу. Патч (patch_glare.js, около 1216 шейдеров) понижает ограничение только для фар:

> $\min(D, 32) \rightarrow \min(D, 6)$ (слагаемые фар; у солнца и запечённых источников остаётся 32)

Ограничение относится к фаре, если между ним и предыдущим ограничением есть чтение цвета этой фары (g_cHeadLightColor[k]). Число переписывается на месте. Шероховатые поверхности, у которых блик никогда не подходит к ограничению, выглядят как раньше.

Headlights in reflections

Карточка: Headlights in reflections. Ночью краска, которой нечего отражать, показывает свет фар.

Там, где отражённый луч из прохода для объектов ничего не находит на экране, материал возвращается к кубмапе неба: ночью это тёмно-синий цвет. Но луч проходит через пучки света фар, которые освещают воздух (игра рисует это отдельным лучом света). Эта сборка модуля чтения для объектов добавляет вдоль отражённого луча свет пучка в воздухе: конус, дальность и цвет берутся из собственных констант игры (CB_GLOBAL_SCENE), так что ночью на капоте виден тёплый свет фар там, где его лучи пересекают пучок. Когда фары выключены или проход не подключён, результат совпадает с обычным модулем чтения и ничего не стоит. С двумя включёнными фарами это добавляет 0.15 мс на целый экран 3840 × 2160 из глянцевых пикселей (замер на RTX 4080); в реальном кадре их гораздо меньше.

Объёмный туман и солнечные лучи

Карточка: Volumetric fog (выключена по умолчанию), со строкой Sun shafts (выключена по умолчанию, экспериментальная).

Как игра рисует туман

Проход тумана (шейдер 0x871EF8CC) проходит каждый луч взгляда рэймарчингом (ray marching): 128 шагов по 0.5 м (0.5 + 0.1 × режим освещения) от начальной глубины. На каждом шаге плотность равна маске тумана, умноженной на спад по высоте; цвет складывается из цвета тени и света фонарей и фар. Отсюда две заметные проблемы. Все пиксели берут выборки на одних и тех же срезах глубины, и под скользящими углами это видно как слои или полосы. И свет не зависит от направления солнца: туман выглядит одинаково и в сторону солнца, и от него.

Джиттер

Каждый пиксель начинает рэймарчинг позже, максимум на один шаг, на значение чередующегося градиентного шума (interleaved gradient noise) n ∈ [0, 1) по координатам рендер-таргета тумана:

> $t_i = t_{start} + (i + n) \cdot \Delta t$

Тогда соседние пиксели берут выборки на разных срезах, и полосы превращаются в мелкий шум, который сглаживает фильтрация буфера тумана.

Рассеяние вперёд

Капли тумана рассеивают свет в основном вперёд, поэтому туман между глазом и солнцем светится. Стандартная модель здесь: фазовая функция Henyey-Greenstein, где θ это угол между лучом взгляда и направлением света, а g это анизотропия:

> $p_{HG}(\theta) = (1 - g^2) \,/\, (4\pi\,(1 + g^2 - 2g\cos\theta)^{3/2})$

Патч умножает рассеянный в луч свет (а не плотность, так что непрозрачность тумана не меняется) на смесь изотропного рассеяния и рассеяния Henyey-Greenstein, взятую относительно изотропного (1/4π):

> $P = (1 - K) + K \cdot p_{HG}(\theta) \,/\, (1/4\pi),\ \ g = 0.3, K = 0.4 \ \Rightarrow\ P = 1.66 \text{ к солнцу}, 0.77 \text{ от солнца}$

Sun shafts (строка)

Стандартный рэймарчинг нигде не проверяет, заслонено ли солнце, поэтому лучей между деревьями нет. Строка встраивает в каждый шаг рэймарчинга вспомогательную функцию (fog_sun.hlsl), которая добавляет рассеянный в луч свет солнца:

> $L_{sun} = 0.12 \cdot C_{sun} \cdot S(P) \cdot \text{cloud}(P) \cdot p_{HG}(\theta;\, g = 0.45) \,/\, (1/4\pi)$

Это значение входит в сумму рэймарчинга с плотностью шага, так же как стандартный цвет. S(P) это карта теней солнца в точке шага, прочитанная так же, как её читают материалы самой игры: глубина вида выбирает каскад (по дистанциям разбиения), точка проецируется матрицей этого каскада в

атлас, и берётся одна билинейная выборка со сравнением. cloud(P) это карта теней от облаков (1 – её красный канал в координатах карты для этой точки).

Ограничение: туман рисуется без знания того, что стоит перед каждым шагом, поэтому при низком солнце лучи видны и поверх грузовика и других предметов рядом с камерой. Поэтому строка экспериментальная и выключена.

Тонмаппинг, блум и цветокоррекция

Карточки: Tonemap, Bloom soft cut, Photo grade. Три шага в конце кадра, в шейдерах постобработки игры (common_pc_sm50_hdr.sdc) и в её цветовых таблицах.

Тонмаппинг

Сцена рендерится в линейном свете, значения идут от 0 и намного выше 1. Кривая тонмаппинга переводит их в диапазон от 0 до 1, который показывает экран. Игра использует кинематографический (filmic) оператор в форме Hable, с параметрами для каждого состояния дня (от TonemapA до F и белая точка W в файлах состояний дня):

> $f(x) = (x\,(A\,x + C\,B) + D\,E) \,/\, (x\,(A\,x + B) + D\,F) - E \,/\, F, \;\; \mathrm{out} = f(x) \,/\, f(W)$

Тонмаппинг Next Gen это реконструкция шейдера игры с другой кривой (CURVE 3): тот же оператор с выключенным членом «носка» (toe) и без белой точки, три константы подобраны под целевой отклик:

> $y = (x^2 + 0.20\,x) \,/\, (x^2 + 0.19\,x + 0.32), \;\; x = 0.18$ в среднем сером

Экспозиция подогнана под собственную кривую игры, так что значение сцены, которое каждое состояние дня переводит в средний серый, по-прежнему попадает туда же: каждое состояние сохраняет заданную авторами яркость, а меняются только контраст, «носок» и «плечо» (shoulder). Замеры на каждую фотографическую ступень (стоп) вокруг серого (в 8-битном выводе sRGB):

	кривая игры	Next Gen
контраст (наклон) в среднем сером	0.91	1.22
вывод на три ступени ниже серого	43	34 (тени глубже, детали сохраняются)
вывод на две ступени выше серого	191	216 (светлые участки ярче, плавно уходят без обрезки)

Вес текстуры блума в итоговом сведении задан 60 % (BLOOM_SCALE 0.6).

Bloom soft cut

Блум добавляет свечение вокруг ярких предметов. Проход «колена» (knee) в игре (0x52879E18) взвешивает каждую выборку по тому, насколько она превышает порог:

> $k = \max(L \cdot \mathrm{exposure} - \mathrm{threshold}, 0), \;\; w = 10 \cdot (0.1\,k)^{\gamma}, \;\; \gamma = \mathrm{BloomGamma} = 0.6$

Показатель степени меньше 1 поднимает слабые значения: снег днём (k ≈ 1.4) получает w ≈ 3.1, и это и есть серая дымка над яркими поверхностями. Патч оставляет этот код и умножает каждый вес на мягкое колено:

> $w' = w \cdot k^2 \,/\, (k^2 + 3^2)$ (снег × 0.18, небо × 0.4, солнце и фары ≈ × 1)

Настоящие блики, у которых k намного выше колена, по-прежнему дают блум.

Photo grade

Игра завершает картинку цветовой таблицей для каждого состояния дня: это LUT 16 × 16 × 16 (в boot.pak), выборка из которой берётся по цвету в кодировке дисплея после тонмаппинга. Четыре таблицы покрывают 161 дневное, вечернее и утреннее состояние (day_us_01 у 64 из них, тождественная у 44, day_us_12 у 34, day_us_18 у 19). Цветокоррекцию замерили на четырёх парах скриншотов: собственные кадры игры против нейросетевых рендеров тех же сцен. Нейросетевые кадры убирают от 13 до 21 % цвета, больше всего из зелёных (× 0.70) и осенних оранжевых (× 0.75), меньше из небесно-голубого (от × 0.86 до 0.9), и чуть опускают белые. Цветокоррекция применяется к каждому выходному текселю таблицы:

Цветокоррекция также убирает грязь на объективе, которую рисует игра. Последний проход постобработки накладывает поверх картинки текстуру мягких дисков (env_camera_bokeh, 1024 × 512), масштабированную светом в месте, куда она ложится, поэтому на ярком небе и облаках она видна как размытые круги, которые следуют за камерой. Тексели этой текстуры записаны нулями (чёрный, без альфы), так что проход ничего не добавляет; Restore и выключенная цветокоррекция возвращают её.

> $\mathrm{LUT}'(c) = \mathrm{grade}(\mathrm{LUT}(c))$

Так собственный вид игры остаётся в основе, а серые остаются серыми. Строка Strength масштабирует цветокоррекцию (100 % это замеренная). Файл таблицы: 82 байта заголовка, затем 4096 текселей в порядке B, G, R, A, где быстрее всего меняется красный, затем 6 байт; записи хранятся без сжатия, поэтому boot.pak переписывается с теми же размерами.

Кубические карты отражений из фотографий

Карточка: Reflection cubes. Нарисованные кубы неба, которые отражаются в краске и хроме и из которых берётся фоновое освещение, заново сделаны из фотографий для каждого региона и времени суток.

Что такое кубическая карта отражений в SnowRunner

Каждое состояние дня указывает один куб (EnvMap="env/day_al_01__d_ggxrefl.tga"; в boot.pak это текстура .pct). У кубической карты отражений (кубмапы) шесть квадратных граней по 256 × 256 текселей в порядке Direct3D +X, −X, +Y, −Y, +Z, −Z, сжатие BC7 и 8 мип-уровней до 2 × 2, каждый из которых простое уменьшение предыдущего. Материалы делают выборку на мип-уровне √roughness × 8 + 0.5, так что мип-уровни соответствуют всё более шероховатым отражениям. Sky ambient и Bounce light берут последний уровень как цвет неба и земли. В игре несколько нарисованных кубов общие для всех регионов. Next Gen делает 25 из 26 кубов, которые указывают состояния, из 21 фотографической панорамы (CC0, Poly Haven; в гараже остаётся игровой куб).

От панорамы к шести граням

Панорама в равнопромежуточной проекции (equirectangular) переводит каждое направление в точку картинки 2:1. Для направления d = (x, y, z), где y направлена вверх:

> $$\varphi = \operatorname{atan2}(x, z),\quad \theta = \operatorname{asin}(y),\quad u = \tfrac{1}{2} + \varphi \,/\, 2\pi,\quad v = \tfrac{1}{2} - \theta \,/\, \pi$$

Каждый тексель грани задаёт направление (центр текселя на грани куба вокруг начала координат, нормализованный); make_cube.js берёт панораму в 3 × 3 точках внутри текселя и усредняет их, чтобы на гранях не было алиасинга. Источник в формате Radiance HDR (RGBE: общая экспонента на тексель), свет линейный.

Экспозиция, цвет и яркость игры

У каждой фотографии своя экспозиция. Куб масштабируется так, чтобы его верхняя половина (небо) пришлась на яркость неба стандартного куба, яркость в sRGB (SKY_MEAN). Настройки отдельно задают небо прямо над головой и землю отдельно от неба, а вечерние сумерки, рассвет или ночь приводятся балансом белого к оттенку игрового куба для этого часа. Затем тексели кодируются в sRGB, как у игры, и мип-уровни строятся простым усреднением (box filter): каждый тексель мипа k + 1 это среднее четырёх текселей мипа k.

Кодирование BC7

BC7 хранит каждый блок 4 × 4 в 16 байтах. Инструмент использует режим 6: одно подмножество, две конечные точки RGBA по 7 бит на канал плюс у каждой общий младший бит и 4-битный индекс на тексель, который выбирает одну из 16 точек на отрезке между ними. Для блока текселей c_i индексы выбираются проекцией каждого текселя на этот отрезок, затем конечные точки A и B уточняются методом наименьших квадратов для этих индексов:

> $$\text{минимизировать } \Sigma_i \left| c_i - ((1 - w_i)\, A + w_i\, B) \right|^2,\quad w_i = \text{index}_i \,/\, 15$$

что сводится к линейной системе 2 × 2 на канал. Общий младший бит берётся тот, что лучше всего округляет все четыре канала, но если альфа конечной точки равна 0 или 255, она сохраняется точно:

когда бит подбирался по цвету, альфа ясного неба выходила где 1, где 0, кольцами вдоль градиента цвета, и множители ночных состояний (× 3 на дальних полосах облаков Мичигана) рисовали их контурами вокруг облаков (видно в игре). 138 байт заголовка файла и 6-байтовый хвост копируются из стандартного куба, так что игра видит ровно ту раскладку, с которой поставляется.

Правило зимних кубов

Игра назначает кубы состояниям по семействам, и два её выбора перестают работать, когда кубы снежных семейств становятся настоящими фотографиями снега. Кольский полуостров и Юкон дают ясным дням 1 и 2 летний куб Мичигана и только дню 3 снежный куб; по этому правилу ясный день снежного региона, который указывает летний куб, получает снежный куб, как у других ясных дней этого региона. А несколько состояний бесснежных регионов указывают снежный куб (погодный вариант Висконсина, вечерние сумерки Северной Каролины); они получают куб, который используют другие состояния того же вида в их регионе. Движок применяет это к initial.pak всякий раз, когда в boot.pak лежат новые кубы.

Ночное небо, яркие звёзды и небо

Карточки: Night sky, Brighter stars, Skies.

Night sky: настоящая звёздная карта

Звёзды игры это одна текстура (4096 × 1024), натянутая на небо, и каждая звезда шириной в тексель. У Next Gen это Deep Star Maps 2020 от NASA в 8192 × 2048: точечные звёзды с яркостью по каталогу и свечение Млечного Пути. Два фотографических ночных неба Скандинавии, Кольского полуострова и Квебека (по одной картинке, это фотографии северного сияния) получают те же звёздные точки, нарисованные поверх.

Свечение слабое: на краю галактики его свет составляет от 1 до 3 из 255 шагов альфы, и каждый шаг прибавляет половину и больше, а множитель слоя (до × 5) показывал это плоскими островками с резкими краями вдоль полосы (видно в игре на ночном небе Вашингтона). Поскольку шейдер взвешивает цвет по альфе, слабое свечение хранится в обоих каналах: альфа округлена вверх, а серый несёт остаток, так что свет меняется шагами в alpha / 255 от шага. Само свечение под этим заново восстанавливается гладким из самой текстуры (наименьшая альфа в окне 5 × 5, что убирает звёзды, затем усреднение 17 × 17 там, где оно слабое, и 5 × 5 там, где яркое); звёзды и яркое свечение остались как были.

Потом часть этой работы испортил кодировщик BC7 (видно в игре, 2026-10-31): из-за общего младшего бита пустое небо оставалось с альфой 1 и белым цветом, чуть ярче слабого края свечения, так что вдоль Млечного Пути шла тёмная кайма, а шаг альфы он взвешивал как шаг серого, поэтому внутри свечения свет перескакивал на целые шаги альфы и выходили контуры. Звёздная карта закодирована заново (out\skies_build\encode_raw.js с весом альфы 256 и мип-уровнями, которые усредняют свет): пустое небо теперь 0, а свет слабого свечения отклоняется в среднем на 0.003, где раньше отклонялся на 0.9.

Brighter stars: почему уровень надо поднять

Яркость слоя неба задаётся как «g(r; g; b; a) × N»: N масштабирует цвет, альфа равна a / 255, а шейдер неба выводит цвет, взвешенный по альфе. Ночные состояния рисуют слой звёзд с уровнем от × 0.07 до × 0.5, подобранным для широких звёзд стандартной текстуры. Точечные звёзды NASA кладут примерно четверть света в тексель каждой звезды, поэтому при стандартных уровнях они выходят на 1 или 2 нит (самые яркие от 6 до 9) на небе от 0.15 до 0.4 нит; у ярких звёзд альфа текстуры уже упирается в 8-битный потолок. Поэтому карточка умножает N в ночных переопределениях слоя звёзд на один общий множитель (вечерние сумерки и рассвет сохраняют свой уровень там, где регион в это время показывает звёзды, ведь небо тогда светлое).

Фотографическое небо такой множитель принять не может: в одной картинке и фотография, и звёзды, так что N сделал бы ярче и фотографию. Поэтому при его сборке доля звёзд хранится в альфе: 85 там, где тексель целиком фото, и до 255 на звёздных точках. Собственная альфа слоя поднимается втрое (50 → 150): фото остаётся как было (85 × 150 = 255 × 50), а звёздные точки выходят втрое ярче.

Skies: купола и облачные полосы из фотографий

Небо SnowRunner состоит из слоёв на сфере: небесный купол (sphere_sky) и до трёх облачных полос (clouds_rear, clouds_mid, clouds_near), которые вращаются вокруг карты, каждая со своим оттенком в каждом состоянии дня. Карточка (включена по умолчанию с 1.1.0) заменяет 49 таких текстур, во всех

регионах, на сделанные из 25 фотографий настоящего неба: купола, которые make_cube.js строит в размере и среднем цвете стандартной текстуры, убрав солнце фотографии, и облачные полосы. Два ночных слоя, чёрные везде, кроме пятна Млечного Пути, получают настоящий Млечный Путь: полосу вокруг центра галактики, взятую из Deep Star Maps 2020 от NASA через поворот из галактических координат в экваториальные, уложенную вдоль длинной оси стандартного пятна (его моменты дают центр, наклон и разброс), затухающую по эллипсу этого разброса, а её свет переложен на собственное распределение яркости стандартного пятна, квантиль за квантилем. Облачные полосы вырезает из панорам make_clouds.js: полоса это текстура в 8 (или 4) раза шире своей высоты, у которой альфа задаёт форму облаков, а цвет их затенение в сером. Где облако, а где небо, определяется по цвету, ведь ясное небо синее, а облако нет:

> $\mathrm{b} = (\mathrm{B} - \mathrm{R}) \,/\, (\mathrm{B} + \mathrm{R}),\ \ \mathrm{b}_{\mathrm{sky}} = 85\text{-й перцентиль b в ряду},\ \ \text{облачность} = \mathrm{b}_{\mathrm{sky}} - \mathrm{b}$

Альфа это мягкая ступенька по облачности, подобранная так, чтобы средняя альфа полосы совпала с покрытием стандартной полосы; у верхнего и нижнего края полоса затухает, как игровые; цвет это яркость текселя относительно самого яркого облака, так что оттенок состояния действует на неё так же, как на игровые полосы. Альфа размыта по соседям, поэтому мягкий край облака заходит на тексели, которые на фотографии относятся к небу; их серый взят от неба, он темнее облака и ночью был виден как тёмная кайма вокруг каждого облака (видно в игре, 2026-10-23). Усреднение серого по тем же соседям, каждый с весом квадрата своей альфы, этого не убрало: тонкий край на фотографии частично небо и выходил темнее своего облака (219 внутри, 186 на краю, 173 сразу снаружи в ближней полосе), а фильтрация между текселями и каждый меньший мип-уровень подмешивают это. Теперь каждый тексель несёт серый ближайших облаков: каждый серый с весом своей альфы в четвёртой степени (плотное облако учитывается, тонкий край почти нет), просуммированный вниз по пирамиде ячеек 2 × 2 и поднятый обратно, так что облако затухает только за счёт своей альфы. Меньшие мип-уровни тоже усредняют цвет с весом альфы, так что цвет ясного текселя не учитывается там, где рядом облако. На верхнем уровне ясные тексели рядом с облаком были ниже мягкого края на величину от 5 до 19 уровней из 255; теперь они не больше чем на 8 ниже, а в некоторых полосах даже выше него.

Зенит: все столбцы купола сходятся наверху, и игра закрывает купол выше примерно 81 градуса веером треугольников, так что всё, что на фотографии есть наверху, сжимается в клинья вокруг точки с кольцом вокруг неё (видно в игре, если смотреть прямо вверх, а на пасмурном небе в виде полос, сбегающихся к зениту). Начиная с 66 градусов каждый ряд купола, а также облачной полосы, которая заходит так высоко, плавно смешивается со своим средним цветом и альфой, полностью на 86 градусах: небо прямо над головой ровное, как в собственных текстурах игры. Ночной купол Юкона (sky_01a_us_04_night) пока сохраняет первую сборку: пересобранный с записанными настройками, он не совпадает с выпущенной текстурой, поэтому наугад его не пересобирают. Ближнюю облачную полосу, которую используют большинство карт (cloud_03_us_01), вырезают с более широким мягким краем (0.45), её альфа размыта на 3 текселя, а пятна меньше 400 текселей выброшены: ночью мелкие обрывки с резкими краями были видны точками на фоне звёзд. Вокруг солнца фотографии в полосе есть только заливка, без собственных облаков; вырезанная там как ясное небо, она оставляла круглую дыру с резким краем. Теперь этот участок берёт облака полосы с противоположной стороны, через пол-оборота на той же высоте, с плавным переходом по краю заливки.

Двадцать текстур сделаны заново для 1.1.1 после сравнения каждой текстуры неба со снимками из игры (2026-10-26). У ясных дневных куполов Кольского полуострова (sky_01_ru_03_day, sky_02_ru_03_day), самых первых сборок (2026-10-12), было плоское серое пятно на месте убранного солнца, зелёно-розовое кольцо вокруг него и холмы фотографии, нарисованные в небе; теперь это Rustig Koppie и Kloppenheim 05. У слоя над днём 2 Кольского полуострова (sky_03_ru_03_day) была

тёмная выемка у горизонта на месте убранного солнца; теперь это Qwantani Dusk 1 в 8192 × 2048. Пять куполов, которые игра рисует в 2048 × 512, с одним уровнем и 82-байтовым заголовком (cloud_us_02_0044, 0049, 0237, 0331 и cloud_us_19_0001), теперь 8192 × 2048, в родном размере фотографий: в их заголовке новые ширина и высота записаны по смещению 0x10, размер уровня по 0x40 и размер данных по 0x4E, а в хвосте размер файла. Собраны скриптом out\skies_build\domes_111.ps1. cloud_02_us_01 тоже стала вчетверо больше, и make_clouds.js --drop-top целиком выбрасывает каждое облако, которое заходит в верхнюю половину затухания верхнего края: облако это его плотная часть (больше 0.35 до затухания, чтобы тонкая дымка не соединяла его с облаками ниже) вместе с мягкой каймой. Игра показала ещё одно облако, срезанное ровно по прямой, в ясный день Кольского полуострова (2026-10-27): тогда у каждой полосы кучевых облаков измерили долю плотных облаков на 8 % ниже верха (cloud_01_us_01 12 % ширины, cloud_01_ru_02 13 %, cloud_01_us_02 48 %, cloud_02_us_02 31 %, cloud_03_us_02 26 %) и сделали их заново с --drop-top, cloud_01_ru_02 и cloud_02_us_02 в 4096 × 512. В cloud_01_us_02 почти каждое облако Sunflowers доходило до верха, поэтому теперь это Day Sky HDRI 012 B от ambientCG.

cloud_02_ru_02 и cloud_02_ru_03, задняя полоса Кольского полуострова и Таймыра, были простыми белыми облаками без затенения в 1024 × 128 текселях, так что вечерний оттенок освещал их как плоские вырезки, а облако, упиравшееся в верхний край полосы, обрывалось прямой линией; к тому же обе были одной и той же картинкой. Теперь это две фотографии низких кучевых облаков от ambientCG (Day Sky HDRI 065 B и 040 B, CC0) в 4096 × 512, вырезанные с меньшим покрытием, чем у игры (0.24 и 0.13), чтобы тонкая дымка фотографий не попала в полосу. ambientCG выкладывает панорамы в OpenEXR со сжатием PIZ; out\skies_build\exr2hdr.py (Python с пакетом OpenEXR) переписывает их в Radiance .hdr, который читает make_cube.js. Размер текстуры игра читает из её заголовка, в котором нет имени, поэтому полоса берёт заголовок другой полосы нужного размера. make_clouds.js принимает --top-edge, долю высоты, на которой затухает верхний край (по умолчанию 0.16, как в игре): при значениях от 0.4 до 0.45 облако, доходящее до края, вместо обрыва истончается. cloud_03_ru_02 (простые белые пятна) теперь Sunflowers в 4096 × 1024, cloud_03_us_01 Wasteland Clouds, а cloud_04_ru_03, серый слой дня 1a Кольского полуострова и Амура и шести состояний Зурдании, теперь Aristea Wreck: раньше это была Kloppenheim 07, та же смазанная ночная фотография, что и в пасмурном куполе Кольского полуострова ниже. Каждую фотографию подбирали так, чтобы ни одно состояние дня не показывало её дважды. Собраны скриптами out\skies_build\bands_111.ps1 и build.ps1.

Ближние полосы пасмурных дней Мичигана (cloud_03_ru_02 в дне 3a, cloud_03_us_01 в днях 1a и 2a) окрашены с множителем × 0.3, и вырезанные кучевые облака с затенёнными низами в цвете выходили непрозрачными тёмно-синими фигурами с резкими краями, вытянутыми вверх прямо над головой (видно в игре на Black River). Собственная ближняя полоса игры просто белая, а её альфа это свет облаков: освещённый верх непрозрачен, затенённый низ прозрачнее, так что тёмный оттенок даёт мягкую серую пелену. make_clouds.js --lum строит полосу именно так: альфа это мягкая ступенька по облачности, умноженная на яркость текселя относительно самых ярких облаков полосы, а цвет белый. cloud_03_ru_02 теперь Kloofendal Overcast, мягкий облачный слой, а cloud_03_us_01 сохраняет Wasteland Clouds, обе с --lum. Самая слабая альфа каждой полосы, меньше 0.04, тоже уходит в 0 (--floor). Пятнадцать полос, собранных make_clouds.js, сделаны заново описанным выше кодировщиком (out\skies_build\bands_112.ps1); остальные сохраняют свои тексели.

Солнце: игра рисует своё, поэтому солнце фотографии убирается вместе с его ореолом. Ореол измеряется как усиление относительно неба на противоположной стороне, градус за градусом от солнца, и делится обратно. Усиление для каждого градуса берётся из кольца текселей на этом угле, отсортированных по яркости, начиная с тех, что на четверти пути вверх: облака рядом с солнцем освещены им и ярче неба, а медиана кольца, наполовину закрытого ими, слишком сильно опускала небо рядом с ними, и вокруг пятна выходило тёмное кольцо. Усиления плавно меняются от градуса к

градусу, потому что скачок был виден как цветное кольцо. Сам диск, от 2 до 6 градусов, где ореол ярче неба больше чем в 30 раз, не содержит ничего пригодного и заполняется с переходом по краю в 4 градуса. Первая заливка переносила цвет каждого направления прямо внутрь от кольца снаружи, и диск рядом с облаком сходился в центре вертушкой из облака и неба с кольцом вокруг (видно в игре в ясный день, 2026-10-23, где это походило на зенит, но стояло на высоте солнца в 48 градусов). Теперь это мембрана, натянутая на это кольцо: на краю каждое направление сохраняет цвет, который там есть, а к середине разницы затухают, мелкие быстрее, до среднего по кольцу. Поверх неё ложится детализация участка такого же размера на той же высоте, отстоящего на угол от 60 до 300 градусов по кругу неба, как его яркость относительно собственной мембраны этого участка (от половины до двух раз), так что облачный слой продолжается сквозь диск; из этих участков берётся тот, чьё кольцо меняется больше всего похоже на кольцо диска. Под облачной пеленой свечение принадлежит самой пелене и остаётся, а заливка поднимается вместе с ним к середине, ровно от края диска внутрь: ровная заливка под свечением была видна как тёмное пятно.

Там, где заливка всё ещё читалась в игре как пятно (мягкий размытый диск на месте солнца, видно 2026-10-23), и в зените это место теперь перерисовано моделью изображений Google Gemini. engine\tools\sky_edit.js вырезает плоские виды собранного купола: один с центром в прежнем положении солнца, шириной 50 градусов с кругом в 10 градусов, и один прямо вверх, шириной 110 градусов с кругом в 35. Каждый отправляется модели с кругом, нарисованным красным, и запросом перерисовать только внутри него и убрать сам круг. Команда apply возвращает ответы на место. Цвет каждой картинки подгоняется к цвету самого купола сразу за кругом (модель немного сдвигает всю картинку: три ответа для Зурдании отличались там на величину от 1 до 2 уровней из 255), картинка берётся полностью до половины градуса за кругом и затухает на протяжении 4, сначала вид зенита, так что правка солнца остаётся последней там, где они встречаются. Затем шапка: от 64 градусов вверх смешивание к одному цвету, полностью на 80, медиане неба между 76 и 80 градусами (их среднее, когда облако заходило в кольцо, оставляло более светлый диск, а смешивание на протяжении 7 градусов всё ещё показывало свой край). Каждый уровень кодируется заново из отредактированного, а рядом с каждым видом записываются его версии до и после. Первые два: ясный день Зурдании (cloud_04_ru_02_day_01, солнце на 48 градусах и зенит) и её пасмурный день (cloud_04_ru_02_day_02, зенит), в игре с 2026-10-23. Купол, собранный заново из панорамы, теряет свои правки, пока на нём снова не запустят apply; виды, ответы и записи о них хранятся в out\gemini.

Три пасмурных купола сделаны заново после того, как в игре они выглядели невыразительно (2026-10-24), тогда как у игры они тяжёлые, грозовые: day__2a Кольского полуострова, который использует и его рассвет (sky_02a_ru_03_day), вышел ясным небом с мелкими облаками и круглой дырой на месте убранного солнца фотографии; day__3a Кольского полуострова (sky_03a_ru_03_day) вышел тонкими полосами, сбегающимися к одной точке у горизонта; пасмурный день Зурдании (cloud_04_ru_02_day_02) вышел ровным серым. Теперь они сделаны из Overcast Soil (оба для Кольского полуострова, у day__2a повёрнут на 180 градусов, чтобы два дня не показывали одни и те же облака) и Mud Road. Kloppenheim 07, которая сначала стояла у day__2a, это ночная фотография: облака сдвигались за долгую выдержку и выглядели в игре смазанными полосами (рассвет Кольского полуострова, 2026-10-26). Пасмурные фотографии плоские (все восемь чисто пасмурных небес Poly Haven помечены как низкоконтрастные), поэтому make_cube.js sky принимает --contrast k, который растягивает свет каждого текселя вокруг среднего геометрического яркости небесной половины, $L' = G\,(L / G)^k$, сохраняя цвет, до подгонки среднего цвета: здесь 1.4, 1.4 и 1.6. Солнце из них не убирается: их свечение мягкое, как в собственных пасмурных текстурах игры. Собраны скриптом out\skies_build\overcast.ps1.

Северное сияние: в двух слоях, где оно есть (cloud_01_ru_03 и cloud_01_us_11, второй зеркальная копия первого с точностью около 6 уровней из 255), оно было синим и фиолетовым, а зелёным только вдоль нижнего края и в одном завитке. Настоящее сияние в основном зелёное, это свет

кислорода на 557.7 нм, часто с красным над ним, где кислород выше и разреженнее (630 нм). Его вырезали квадратом на чёрном фоне, 2080 текселей шириной, и Gemini перерисовал его в этих цветах, сохранив форму и занавеси. Ответ возвращается масштабированным к квадрату, его чёрный уровень (3 из 255, шум JPEG, который затянул бы небо пеленой) вычитается, цвет усиливается в 1.6 раза вокруг собственного серого каждого текселя, а общий свет приводится к свету стандартного сияния (усиление 0.78). Вторая текстура это зеркальная копия первой, и обе сохраняют стандартную альфу. Ответ, вырезка и скрипты хранятся в out\gemini\aurora.

Каждая текстура ставится вместе со своим .pct_header, который игра держит рядом с каждым .pct: это заголовок .pct, у которого последняя запись (тип 0xff, длина данных после неё) превращена в тип 1 с длиной самого заголовка, как во всех 349 парах стандартного boot.pak. До 2026-10-23 инструменты записывали начало .pct как есть, и купола, уменьшенные до стандартного размера (4096 × 1024), сохраняли старый заголовок от 8192 × 2048; теперь все пары согласованы.

Дым: свечение, форма, мягкие края

Карточки: Smoke glow (строка More kinds of smoke), Smoke shape and softer edges.

Как игра освещает клуб дыма

Выхлоп, пыль, пар и дым это спрайты: плоские текстурированные квадраты, всегда повёрнутые к камере. Игра освещает каждый одним значением солнца и одним фоновым значением (ambient) на вершину: это либо запечённая карта освещения сверху, либо цвет солнца, умноженный на коэффициент видимости. Ничто не зависит от положения солнца относительно камеры, и у спрайта нет формы: клуб дыма это плоский диск света.

Свечение: рассеяние вперёд

Дым рассеивает солнечный свет в основном вперёд, поэтому на фоне низкого солнца он светится, часто ярче неба за ним. Вспомогательная функция, встроенная в вершинные шейдеры частиц, умножает солнечный свет клуба на

$$f = \min(1 + 0.15 \cdot \max(4\pi\, p_{HG}(\mu;\, g = 0.7) - 1,\, 0),\, 2)$$

где μ это косинус угла между лучом взгляда и направлением солнечного света, а p_{HG} это фазовая функция Henyey-Greenstein (см. главу о тумане). $4\pi\, p_{HG}$ равно 1 для изотропного рассеяния, поэтому f = 1 начиная с 90 градусов от солнца и дальше, а к солнцу множитель растёт (если смотреть прямо на солнце, $4\pi\, p_{HG} \approx 18.9$, и срабатывает предел 2). Множитель стоит на солнечном члене, поэтому тест теней в пиксельном шейдере, запечённая тень карты освещения, ночное затухание и туман по-прежнему действуют. Частицы, которым вершинный шейдер вообще не даёт солнечного света, получают цвет солнца, умноженный на f − 1, то есть одно свечение.

Вершинные шейдеры частиц делятся на несколько семейств, и патчу нужно найти солнечный член в каждом: карта освещения, декодированная из одной компоненты регистра; прибавленный цвет солнца; цвет солнца, умноженный на видимость (плюс фоновый свет). Из 205 вершинных шейдеров 162 получают свечение по умолчанию. Строка More kinds of smoke добавляет 8, у которых карта освещения декодируется в компонентах x, z и w регистра (170 из 205). Из остальных 35: 10 это клубы с картой нормалей (освещаются в пиксельном шейдере), 22 не выводят свет, а у 3 нет констант сцены.

Форма: псевдонормаль из альфы

Часть в пиксельном шейдере (smoke_shade, в 80 освещённых клубах со смешиванием без собственной карты нормалей) даёт каждому пикселю нормаль мягкого шара. Её направление на экране это градиент альфы клуба (наружу, туда, где альфа спадает), а наклон задаёт сама альфа: тонкий край смотрит вбок (синус наклона 0.9 при альфе 0), плотное ядро смотрит в камеру. Если f это нормаль плоского спрайта (к камере), а L это направление на солнце, освещённый цвет умножается на

$$k = (1 + 0.3\,(n \cdot L - f \cdot L) + 0.15\,(n_y - f_y)) \times (1 - 0.12 \cdot \mathrm{smoothstep}(0.5,\, 1,\, \alpha))$$

Сторона к солнцу получает то, что теряет дальняя, верх светлее низа (свет неба сверху), плотные ядра чуть темнее (свет теряется внутри клуба). Клуб, который виден анфас в середине, сохраняет

стандартное освещение. Масштабируется только освещённая часть, смешивание с туманом после неё остаётся: out = c + (k − 1)(c − fog.a · fog.rgb).

Белый дым рисуется тоньше: с игровой непрозрачностью он выходит слишком белым и слишком плотным. Там, где собственный цвет клуба (его альбедо, до освещения) имеет яркость от 0.1 до 0.3, его непрозрачность меняется от игровой до 0.6 от неё. Чёрный выхлоп сохраняет игровую непрозрачность.

Мягкие края

Там, где спрайт пересекает землю, видна жёсткая линия. Игра растворяет спрайты по глубине позади них; в 110 шейдерах, которые это делают, затухание теперь идёт на вдвое большем расстоянии, поэтому клубы мягко сходятся с землёй.

Более чёткий дым и частицы

Карточка: Sharper smoke and particles.

40 текстур частиц игры (файлы sfx_*: дым, пыль, искры, брызги) заменены версиями с удвоенной шириной и высотой, которые автор увеличил с помощью NVIDIA DLSS 5 Visual Enhancer и сохранил в собственном формате текстур игры (.pct, с его .pct_header). Каждая заменяет свою запись в boot.pak и хранится так же, как её хранит игра. Они остаются ассетами Saber Interactive и поставляются только для использования с игрой. Цена: вчетверо больше памяти под эти 40 текстур.

Погода, мусор и брызги

Карточки: Weather (девять строк), Debris and splashes. Обе правят собственный XML игры в initial.pak: что несёт каждое состояние дня и как ведёт себя каждый тип частиц.

Что несут состояния игры

Игра проводит каждый регион через его состояния дня и их погодные варианты (day__1, day__1a, на новых картах до day__1e). Состояние может нести дождь или снег, <WeatherParticles Type MinIntensity MaxIntensity IntensityChangeInterval>, интенсивность которых каждый интервал заново выбирается в пределах диапазона, и движущиеся тени облаков, <CloudShadowMap Map Scale Speed Direction>. В стандартной игре дождь или снег в вариантах есть у 16 регионов, погода в сумерках и ночью есть только у снежных регионов, а тени облаков есть только у 39 из 199 состояний.

Строки

Строка	Что записывает
shadows	тени облаков в каждом состоянии дня, где их нет, как в Зурдании (масштаб 6, скорость 2, направление (1; 0.5)): лёгкая карта для обычных состояний, средняя для вариантов, плотная для сильного дождя или снега
showers	дождь и снег то усиливаются, то стихают: диапазон интенсивности (min, max) становится (0.3 min, max), пики остаются игровыми (1.2 max до версии 1.1.1: сильный снег тормозил)
evening	регионы с дождём днём получают лёгкую морось в сумерках и на рассвете (от 0 до 0.15) и ночью (от 0 до 0.2)
horizon	слой облаков на горизонте, закомментированный в 17 файлах неба, включён, с переопределениями для собственных состояний файла
far	дождь рисуется в 1.5 раза дальше от камеры (32 м → 48 м); по умолчанию выключено. Снег сохраняет игровую дальность: если рисовать его так далеко, хлопья заполняют примерно втрое больший объём вокруг камеры, а сильный снег в игре сильно тормозил
fireflies	светлячки ночью в дождливых регионах: новый тип погоды с игровым спрайтом светлячка, рисуется до 90 м при интенсивности от 0.0005 до 0.001
pollen	пыльца, летающая в ясные дни тех же регионов, рисуется до 64 м при интенсивности от 0.01 до 0.05
drift	облака, которые игра держит неподвижными, медленно поворачиваются
land	земля рисуется так далеко, как доходит дальний туман: где состояние скрывает рельеф (TerrainHideDist) ближе, чем кончается туман, дистанция поднимается до конца тумана (по умолчанию включено)

Светлячки и пыльца это типы погоды из более позднего движка игры (Expeditions); здесь это два новых файла в classes\weather. Каждый записываемый атрибут встречается в собственных файлах игры.

Debris and splashes

Типы частиц это небольшие XML-файлы (по одному <CombineParticles> в каждом). Карточка применяет по одному правилу на семейство:

- Обломки (листья, трава, ветки, грязь, камни из-под колёс): в стандартной игре они исчезают через 3 или 4 с. Теперь они живут от 30 до 40 с (LifeTime 30, Random 10) и сталкиваются с землёй, грузовиком и его колёсами, с радиусом 1 см, чтобы кусочек ложился плашмя, и с небольшим отскоком. Семена одуванчика парят в воздухе, поэтому им только продлена жизнь (от 12 до 16 с).
- Брызги (грязь, вода, снег, выбросы земли): всплеск выбрасывает в D раз больше капель (Subparticles Density, по умолчанию D = 4, от 12 D до 64 D), капли лежат вдвое дольше, сам всплеск длится вдвое дольше и оставляет на грузовике вдвое больше следов грязи и воды.
- Клубы (выхлоп, пыль, пар) висят в воздухе в полтора раза дольше (LifeTime × 1.5).
- Снежные брызги длятся втрое дольше, а их капли падают с половинной гравитацией при 0.6 от сопротивления воздуха и живут 15 с: лёгкий снег висит в воздухе.
- Водяные брызги: то, что выбрасывает колесо, буксуя в воде (water_splash_small, water_flow), и капли от этого (water_splash__subparticles, water_droplet__subparticles) летят дальше и стали крупнее. В стандартной игре брызги останавливаются в пределах полуметра: их сопротивление воздуха 3.0. Сопротивление воздуха брызг и их капель теперь 0.4 от игрового (с 3.0 до 1.2, с 1.0 до 0.4, с 1.5 до 0.6), спрайты брызг в 1.5 раза крупнее и живут втрое дольше, капли в 1.5 раза больше по масштабу. Сила броска колеса остаётся игровой; при линейном сопротивлении воздуха k бросок уносит на v / k, так что тот же бросок летит примерно в 2.5 раза дальше.

Цена: одновременно живёт больше частиц, значит, больше работы для отрисовки; по умолчанию D равно 4.

Детализация окружения и дальность травы

Карточки: Scenery detail (строки: ближние переключения, скрытие, тени), Grass reach.

Уровни детализации

Каждый меш несёт свои упрощённые версии и дистанции, на которых игра на них переключается (LOD). Многие меши переключаются на расстоянии от 10 до 15 м от камеры. С GTAO и более чёткими тенями это переключение видно как скачок затенения на фиксированной дистанции, особенно на скальных выходах. Меши лежат в shared.pak (2 ГБ) как «[meshes]\models_<name>» и «[meshes]\plants_<name>»: у каждого 32-битный размер, XML-заголовок, затем двоичные данные. Переключения записаны в заголовке:

> <MeshLod Distances="10, 25, 45" ... />

Если первое переключение меша ближе 40 м, его переключения поднимаются как минимум до 40, 70, 110 и 160 м; значение, которое уже дальше, остаётся. Набор карточки либо «nature» (скалы, деревья и кусты по имени; пни, ветки, посевы, руины, стены и знаки исключены), либо все меши с переключениями. Две дополнительные строки убирают дистанцию, за которой меш больше не рисуется (HidingDistance), и тег, который обрывает тень мелкого объекта или запрещает ему её отбрасывать (MeshShadow), для растений или для всего.

XML должен сохранить свою длину в байтах, чтобы двоичные данные после него и префикс размера остались байт в байт: разницу забирают или отдают пробелы между тегами.

Пересжатие только изменённого

shared.pak это zip; многие его записи сжаты deflate, а XML меша стоит в начале его потока. Поток deflate состоит из блоков, у каждого свои таблицы Хаффмана. Пересжимать меши целиком было бы долго; вместо этого инструмент перекодирует только блоки с XML-заголовком, а остаток исходного сжатого битового потока дописывает как есть, со сдвигом на новую битовую позицию (блок не обязан начинаться на границе байта). Результат снова распаковывается и проверяется до того, как пак будет записан. Неизменённые записи копируются как есть, несжатые (stored) остаются несжатыми.

Дальность травы

У мешей травы нет переключений; игра гасит траву по дистанции для каждого типа травы, в initial.pak:

> <GrassBrand FadeDistances="7, 17, 30"> ступени плотности, затем конец травы

Большая часть травы исчезает к 30 м. Карточка умножает дистанции на свой множитель, с одним ограничением: игра рисует траву только примерно до 64 м от камеры, а ступень, заданная дальше, уже не затухает, и трава на краю появляется сразу целиком (обнаружил Milliath). Поэтому ступень n из k заканчивается не дальше, чем

> $$d'_n = \max(d_n, \min(f \cdot d_n, 60 - 5\,(k - 1 - n)))$$

что даёт 21, 51, 60 м при 3 и 14, 34, 60 м при 2 для обычного «7, 17, 30»; ступень, которую сама игра задаёт дальше (у некоторых типов конец на расстоянии от 80 до 150 м), остаётся. Цена: рисуется

больше травинок, и на густых лугах именно в этом настоящая цена карточки.

Дни карт (Map days)

Карточка: Map days. Состояния времени суток и небо какого региона проходит каждая карта.

Где карта называет свои дни

Пак каждого уровня (level_us_06_01.pak) содержит несжатый [scene]\data_for_ssl.bin, который начинается с имён, каждое: 16-битная длина, текст и ноль. Это имя уровня, его описание, затем состояния ночи, рассвета, трёх дней (каждое: список через ; из состояния и его погодных вариантов), заката и файл неба региона. Четыре региона не имеют своих состояний и называют чужие: Мэн называет состояния Юкона и Аляски, Британская Колумбия Мичигана, Австрия Белозерска и Северной Каролины, Вашингтон Северной Каролины. Игра берёт имена как есть: контрольное значение самого файла не сверяется (проверено в игре), и назвать можно любое состояние, известное initial.pak. Неизвестное имя останавливает загрузку.

Правка

Каждый код региона в начале файла заменяется другим той же длины (us_04 → us_01), в собственном файле игрока, на месте. CRC записи в zip обновляется в обоих местах, в локальном заголовке и в центральном каталоге. Для отката ничего не сохраняется: Restore записывает коды игры, которые хранятся в таблице инструмента, и CRC получается прежним. Файл уровня, изменённый другим модом, остаётся как есть. Apply и Restore проверяют, что файлы уровней доступны для записи, только когда собираются их писать (Map days выбрана или установлена), так что файл уровня только для чтения не останавливает запуск, который карты не трогает.

Варианты

Текстуры снега и льда каждого уровня посчитаны по его списку загрузки (в Yellowrock National Forest снега и льда в два-четыре раза больше, чем в The Lowland, который на уровне Мичигана), а сезон каждого региона взят из вики и сверен с игрой (Скандинавия, записанная как весна, там поздняя зима, со снегом). Значение map by map выбирает по уровням: The Lowland дни Мичигана, Yellowrock весну Юкона с метелями, ослабленными до снегопада, Duncan Bay лето Висконсина с его ливнями, North Peak Северную Каролину, Unterfeld Теннесси, Washington осень Мичигана, а его Mountains Зурданию. У каждого целевого региона есть все десять имён состояний, которые использует уровень.

Собственные наборы карты

Две карты Мэна получают собственные наборы, us_91 и us_92, номера, которые не использует ни один регион игры. Набор делается из состояний и неба исходного региона в том виде, в каком они лежат в сборке: файлы под новыми именами, имена состояний внутри файла неба (DayTimeOverride DayTime="day__1_us_04"), переименованные вместе с ними, и погода, изменённая по таблице (Yellowrock: heavy_snow становится light_snow с 0.4 своей интенсивности). Движок добавляет эти 20 файлов в initial.pak, несжатыми, в конец, а их записи каталога ставит туда, где положено по именам; затем вносит каждый файл в список загрузки пака, без которого игра о них не знает (следующая глава). Файл уровня называет набор, только пока initial.pak его содержит: инструмент добавляет набор до записи файлов уровней и убирает его, когда ни один из них его больше не называет. Если обновление игры заменит initial.pak, пока уровень ещё называет набор, статус покажет «part», и следующий Apply это исправит. Restore оставляет initial.pak как есть, пока файл уровня ещё называет набор: если один из двух файлов Мэна изменён другим модом, файлы уровней не возвращаются, и

собственный initial.pak игры, в котором такого набора нет, не дал бы загрузиться другой карте; вместо этого Restore об этом сообщает.

Список загрузки: как пак сообщает игре, что в нём лежит

Часть движка, на которой стоят собственные наборы Map days и любой файл, добавленный в пак.

Первая запись каждого пака, pak.load_list, перечисляет всё, что пак загружает, и в каком порядке: начало, этапы («TEMPLATES load», «CLASSES load», «TEXTURE load», «MESH load»), ресурсы между ними, конец. Файл класса (грузовик, состояние времени суток, небо), который есть в паке, но не в этом списке, игре неизвестен: уровень, который его называет, падает. Это проверено в игре: двенадцать состояний, добавленных в initial.pak без записей в списке, роняли уровень; те же состояния с записями загружались.

Разметка следует разбору из SnowRunnerTools от chase-000:

Часть	Байты
заголовок	i32 1, u8 1, i32 count, i32 3, u8 1
типы	count байт: 5 начало, 1 этап, 2 ресурс, 6 конец; затем u8 1
ожидания	на запись: i32 n, u8 1, n × i32, записи, которых она ждёт (счёт с 0, по возрастанию); затем u8 1
строки	на запись: i32 s, i32 2, s + 2 байта 01, затем s строк (длина u32 и текст)

У ресурса три строки (его имя, его загрузчик, его пак: «<media>\classes\daytimes\x.xml», «cls_loader», «initial.pak») или четыре (текстура в паке уровня: JSON-текст с именем её основы). Правила, которые соблюдают собственные списки игры: ресурс ждёт одну запись (этап перед своей серией или, для граней текстуры, её заголовок); этап после ресурсов ждёт ровно те, которых больше никто не ждёт; этап после этапа ждёт этот этап; файлы классов стоят вместе в байтовом порядке своих имён. Инструмент читает все 73 списка игры и записывает их обратно байт в байт. Добавить файл класса значит: новая запись встаёт туда, где положено по имени среди файлов классов, и ждёт этап перед ними; каждый номер, указывающий на более позднюю запись, увеличивается на один; этап после файлов классов тоже ждёт новую запись.

Логотип Next Gen и запуск без заставки

Карточка: Next Gen logo (строка Start-up).

Логотипы

Надпись NEXT GEN появляется под логотипом игры на пяти текстурах в gfx.pak. Три из них отдельные картинки (титульный экран, первый экран загрузки 3840 × 2160, главное меню) и заменяются целиком. Две это листы из многих спрайтов (меню паузы, 500 × 504, и экран карты, 512 × 508), где должен меняться только спрайт логотипа, чтобы остальные спрайты мода интерфейса остались. Листы в BC7: каждый блок 4 × 4 текселя занимает свои 16 байт, независимо от соседей. Поэтому инструмент копирует ровно те блоки, что внутри прямоугольника спрайта логотипа (кратного 4 текселям), из листа Next Gen в лист, который лежит в паке, а все остальные блоки остаются бит в бит. Лист другого размера или формата остаётся как есть.

Запуск без заставки

Заставка игры не видео: её крутит один ролик Flash (Scaleform) в [gfx]\gfxbundle.gfxbundle из gfx.pak, 35 кадров в секунду. Кадры с 2 по 259 показывают предупреждение об эпилепсии и юридическое предупреждение (7.4 с); скрипт на кадре 259 останавливает ролик и сообщает экрану загрузки, что всё готово (OnSkippableReadyToStart); затем экран загрузки вызывает StartSkippables, который проигрывает логотипы издателей (около 11 с, пропускаются клавишей); в конце OnAllAnimFinished передаёт управление (OnLogoEnd), и появляется загрузочный экран bootup: заставка Next Gen. Строка правит байт-код ActionScript ролика, в самом ролике и в его японском двойнике:

Где	Байты игры	Байты Next Gen
addFrameScript в конструкторе	pushshort 258 (25 82 02)	pushbyte 9, nop (24 09 02): скрипт «готово» срабатывает на кадре 10
StartSkippables (38 байт)	фокус, слушатель ввода, gotoAndPlay("skippable")	nop, затем this.OnAllAnimFinished(): логотипы сразу заканчиваются

Каждая правка сохраняет размер, так что заголовок бандла (в нём перечислены имена и длины роликов, без смещений и контрольных сумм) остаётся корректным; меняется только CRC записи в zip пака. Каждая правка ищется по байтам всего метода в том виде, в каком их поставляет игра; версия игры с другим роликом сохраняет заставку. Замер: титульный экран появлялся через 41 с после запуска с заставкой и через 24 или 25 с без неё (тёплый дисковый кэш). Загрузка за заставкой всё равно должна закончиться; выигрыш это та часть заставки, которая длилась дольше неё.

Движок: сборка, проверка, откат

Что происходит, когда вы нажимаете Apply или Restore. Окно передаёт выбор (отмеченные карточки и их строки) движку, набору инструментов на Node.js, который лежит в папке engine архива.

Паки это zip

Каждый пак это обычный zip: записи подряд, stored или deflate, без zip64 в тех паках, которые трогает Next Gen, дополненный нулями до кратного 4096 байтам размера после конечной записи. Некоторые из них большие (shared.pak 2 ГБ, boot.pak 1.1 ГБ, gfx.pak 0.7 ГБ), поэтому запись копирует неизменённые записи как есть, крупными кусками, и пишет только изменённое. Изменённая запись получает новый локальный заголовок (свой CRC-32 и размеры), центральный каталог получает новые смещения всех записей, а добавленные записи идут после последней.

Всегда от оригинала

Перед первым изменением пака движок сохраняет его копию в %LOCALAPPDATA%\SnowRunnerNextGen (по папке на каждую установку игры) и после копирования проверяет её по хешу. Каждая сборка начинается с этой копии, никогда не с файла в игре: сборка зависит только от оригинала и выбора, поэтому два Apply подряд дают тот же файл, а убрать карточку значит то же, что никогда её не ставить. Несколько карточек могут делить один пак: initial.pak, например, несёт дальность травы, заполняющий свет, яркие звёзды, погоду, мусор из-под колёс и собственные наборы Map days, в одной сборке.

Записать, прочитать обратно, затем заменить

Новый пак пишется рядом со старым (name.tmp), читается обратно и проверяется: те же записи в том же порядке, каждая изменённая запись распакована и сравнена с тем, что планировалось, все остальные записи каталога не изменились, и у каждой 50-й нетронутой записи прочитан и проверен CRC. Только тогда он заменяет старый файл, переименованием после сброса на диск, так что сбой или отключение питания оставляет либо старый файл, либо новый, но никогда половину. Движок отказывается писать, пока запущен SnowRunner.exe; он запрашивает у Windows процесс игры в UTF-8, так что папка игры с буквами вне кодовой страницы консоли (D:\Игры\SnowRunner) тоже распознаётся.

Как движок узнаёт свои сборки

Для каждого пака движок записывает (в небольшой JSON рядом с сохранённым оригиналом) SHA-256 центрального каталога сборки, её размер и что в неё вошло. Хеша каталога достаточно, чтобы узнать файл: в нём CRC-32, размер и смещение каждой записи. Поэтому у пака одно из четырёх состояний:

Состояние	Значение
ours	файл это последняя сборка движка (хеш каталога совпадает с записью); запись говорит, какие карточки в нём есть
vanilla	файл это сохранённый оригинал
changed	ни то ни другое: с тех пор его записал другой мод или обновление игры
missing	такого файла нет

Запись делается в два шага вокруг замены (сначала предварительная, после замены окончательная), так что сборка, прерванная между ними, всё равно распознаётся. Пак, который не может прочитать ни один инструмент (файл zip64, байты после конечной записи, больше 65535 записей: разметка другого инструмента), считается changed, и лог объясняет почему: статус, Apply и Restore продолжают работу со всеми остальными файлами, а не останавливаются целиком. Считается только такая разметка: любой другой сбой инструмента (блокировка, полный диск) останавливает работу, как и раньше, потому что ответ «да» на вопрос, который за ним последует, сохранил бы собственный shared.pak или shader.pak движка как оригинал.

Собственное состояние движка для игры (сохранённые оригиналы, записи) называется по настоящему пути папки игры, так что та же игра, открытая через junction, диск subst или другую букву диска, его находит. Перед первым копированием оригинала движок проверяет, что на диске хватит места, с запасом 256 МБ, а копирование, сорвавшееся на полпути, не оставляет недописанных файлов. На одной игре одновременно идёт только один Apply или Restore, это держит файл блокировки с номером процесса; блокировка, чей номер теперь принадлежит другой программе или которая старше шести часов, перехватывается.

Небеса скачиваются отдельно (архив с небом). Каждое изменение boot.pak собирает его заново из оригинала с уже установленными частями, так что новая версия, распакованная в новую папку без пака небес, всё равно сохраняет небеса: они берутся из boot.pak в игре, собственной сборки движка. Restore убирает небеса первыми. Пак несёт список своих файлов и их размеров (pack.json): пак, недораспакованный архиватором, включая сам список, распознаётся, и Apply не ставит небеса, а предлагает распаковать его заново, вместо того чтобы записывать те текстуры, что есть. Если небеса установлены, а пака нет, ответ «да» на предложение принять изменённый boot.pak как оригинал отклоняется, пока пак не вернётся: небеса последней сборки отличаются от изменений другого мода по собственным файлам пака, и без них вся последняя сборка осталась бы в новом оригинале, вне досягаемости Restore. Текстура или LUT, которые обновление игры переименовало или убрало, пропускаются с предупреждением, а не останавливают каждый Apply на одном и том же шаге, а шейдерный модуль, который шейдеры игры больше не принимают, запоминается как пропущенный, чтобы следующий Apply не пересобирал из-за него shader.pak.

Другие моды и обновления игры: принятие

Когда пак в состоянии «changed», Apply спрашивает. «Take it as the original» принимает файл: движок проходит по его записям, и каждая запись, которая всё ещё содержит ровно то, что записала последняя сборка движка (тот же CRC, тот же размер, те же байты), возвращается к байтам старого оригинала. То, что изменил другой мод, остаётся. Результат становится новым оригиналом, и сборка идёт поверх него. shared.pak с версии 1.1.2 проходится так же (меш-мод вроде RLM, установленный через WinRAR, меняет его): раньше при принятии он брался целиком, и поднятые переключения и дополнения последней сборки оставались в новом оригинале, так что Restore оставлял их в игре, а снятие отметки со Scenery detail их уже не убирало. Restore задаёт тот же вопрос, если shared.pak изменился после последнего Apply, и убирает из него меши Next Gen, прежде чем вернуть его на место. Старый оригинал хранится под именем с датой, пока этот Apply не закончится, затем удаляется: его ничто не читает, а у shared.pak он весит 2.1 ГБ. Restore затем возвращает это модифицированное состояние, вместе с изменениями другого мода. «Leave it out» оставляет файл и его карточки как есть. Файлы, которые сборка добавила в initial.pak (собственные типы погоды, файлы классов дней) и которые всё ещё содержат ровно её байты, в новый оригинал не попадают. Проверенная перепаковка RLM через WinRAR поверх всех частей дала файлы игры плюс файлы RLM и после принятия, и после Restore.

Обновление игры заменяет паки, поэтому после него игра выглядит по-своему, пока Apply не запущен снова, и Apply задаёт тот же вопрос. Обновление игры от 20 октября 2026 (сборка exe

6ab52f5d, новый регион) проверено так на копиях новых файлов: каждая карточка применилась без предупреждений. Движок также узнаёт собственный shader.pak игры по хешу, для июльской версии и этого обновления, так что при первой установке он принимается как оригинал без вопросов. Одна часть SnowRunner Shadows сама читает код игры: правило основы текстур. Игра хранит самые мелкие мип-уровни (mip) текстуры в блоке, начало которого вычисляет из одного значения, 512, а апскейленные текстуры со сторонами вроде 192 загружаются только тогда, когда это значение равно 256. На сборке, для которой DLL сделана (6a607c05), она выставляет его при загрузке по известному адресу. На любой другой сборке она ищет значение, когда игра создаёт устройство Direct3D, до появления любой текстуры: через единственную последовательность инструкций, которая его читает (mov ecx, [value]; mov eax, ecx; cdq; sub eax, edx; sar eax, 1), в коде игры, указывающую на int со значением 512. Если ничего не найдено или читающие инструкции указывают на два разных значения, она не трогает правило игры и пишет об этом в свой лог.

Заметка в папке игры

Рядом с паками движок оставляет SnowRunnerNextGen.json, где перечислены файлы с его изменениями и хеши их каталогов. Если сохранённые оригиналы когда-нибудь потеряются (новая установка Windows, игра на другом диске), движок всё равно видит, что эти файлы не собственные файлы игры; тогда он ничего не меняет и просит сначала проверить файлы игры через магазин, чтобы не сохранить модифицированный файл как «оригинал». Restore убирает эту заметку.

Шейдеры

Первый Apply читает собственный кэш шейдеров игры и готовит из него шейдерные карточки (минуту-другую): каждый инструмент-патчер проходит по стоковым шейдерам, а то, что он сделал, записывается вместе с хешами. Следующие Apply собирают готовые модули для отмеченных карточек и пишут shader.pak за один проход. DLL, hid.dll, копируется рядом с SnowRunner.exe вместе с файлом настроек; hid.dll другого мода сохраняется как hid_chain.dll и по-прежнему загружается.

Тесты

Каждое изменение Next Gen перед выпуском проходит цепочку тестов: тесты окна (карточки, строки, выбор, который оно отправляет), тесты движка на подставных папках игры, собранных из настоящих оригиналов (каждая карточка включена, выключена, изменена другим модом, принята, восстановлена байт в байт), сборка пакета и повторный прогон упакованного движка. Последняя полная цепочка на этой версии прошла 454 проверки.

Известные проблемы и исправления

Что всплыло при написании этого руководства, что исправлено, и ограничения, которые заложены в саму технику, а не ошибки: о них не нужно сообщать как об ошибках.

Исправлено

Что было не так	Что изменилось	Коммит
Справка Map days говорила, что у трёх регионов нет собственных дней; четвёртого, Вашингтона, не хватало.	Справка называет четыре региона и строку Washington.	1c8ad5e, 2026-10-17
README описывал Map days без Вашингтона и без значений map by map, карточку логотипа без строки Start-up, изменяемые файлы без собственных наборов Мэна, а запуск без заставки и файлов уровня Вашингтона.	README их перечисляет.	1c8ad5e, 2026-10-17
Список клавиш в игре в окне (How it works, сравнение в игре) называл F4, F5, F6, F8, F10 и F11, но не F9, которая переключает отражения на мокрой земле.	Список называет F9.	d01a6b5, 2026-10-18
Устаревшие заметки к инструментам: инструмент погоды говорил, что в нём пять частей (их девять); lod_patch.js говорил, что погода по умолчанию это все семь частей (это все части, кроме land, то есть восемь), а в его справке по использованию не было debris-install, days-set и параметров debris= и days= у initial-build.	Заметки и печатаемая справка говорят то, что инструменты делают.	d174ccd, 2026-10-18
Игрок видел в тенях старую картинку, из меню, после смены настройки видео в игре, пока не нажмёт F8 или не перезапустит игру. Копия сцены, которую читают переотражённый свет и отражения, оставалась привязанной, когда копирование прекращалось, и хранила свою последнюю картинку.	Копия привязана только в пределах 250 мс после последней; иначе каждый читатель не получает ничего и сохраняет собственный результат игры. В игре ещё не проверено.	2026-10-21

Каждое исправление прошло полную цепочку тестов (454 проверки), прежде чем было закоммичено. Ни одно из них ничего не меняет в игре.

Ограничения по замыслу

Карточка	Ограничение
Object reflections, Water reflections, Wet ground reflections, Contact shadows, GTAO, Bounce light	Экранное пространство: они знают только то, что видит камера. То, что за кадром или скрыто за чем-то, не отражается, не отбрасывает контактной тени и не даёт затенения; отражения гаснут у краёв экрана.
Object reflections, Wet ground reflections, Bounce light	Они читают прошлый кадр, поэтому отстают на кадр и ничего не имеют для только что открывшихся пикселей.
Bounce light (выключена по умолчанию)	Местами, например на ярком снегу рядом с тёмным деревом, выходит слишком сильным.
Volumetric fog, Sun shafts (выключены по умолчанию)	Туман не знает, что стоит перед каждым шагом, поэтому при низком солнце лучи видны и поверх грузовика и других предметов рядом с камерой.
Water reflections, Sun glitter in shade (выключено по умолчанию)	Там, где это пробовали, собственная проверка теней игры считала воду освещённой, поэтому ничего не менялось.
Sun glow through waves	8 из 20 шейдеров рек не читают карту теней солнца и свечения не получают.
Shadow edges, дизеринг швов	Полоса, где смешиваются каскады, полагается на TAA игры; детали мельче текселя теней восстановить нельзя.
Map days, map by map для Maine	Собственные наборы Мэна лежат в initial.pak. Если обновление игры заменит его, пока файлы уровня Мэна ссылаются на эти наборы, Мэн не загрузится, пока снова не выполнится Apply.
Next Gen logo, Start-up	Игра по-прежнему грузится за заставкой; выигрыш во времени это та часть вступительного ролика, которая длилась дольше загрузки (около 16 с в тесте).

Часть 5. Что меняется на вашем ПК

Всё в папке игры **preload\paks\client**, и только те файлы, которые нужны отмеченным карточкам:

Файл	Что в него кладёт Next Gen
shader.pak	шейдерные модули: тени, затенение, свет, отражения, вода, туман, дым, тонмаппинг
shared.pak	Scenery detail
initial.pak	Grass reach, заполняющий свет, Brighter stars, Weather, Debris and splashes и собственные дни Мэна (Map days, map by map)
boot.pak	Photo grade, Sharper smoke and particles, Night sky, Reflection cubes, Skies
gfx.pak	Next Gen logo и запуск без вступительного ролика
файлы уровней	с Map days несколько имён в файлах уровней Мэна, Британской Колумбии, Австрии и Вашингтона, изменённые на месте

В папке с SnowRunner.exe (в Steam это Sources\Bin): **hid.dll**, то есть SnowRunner Shadows, и его настройки, **SnowRunnerShadows.ini**. hid.dll другого мода продолжает загружаться под именем hid_chain.dll.

Копии исходных файлов хранятся в **%LOCALAPPDATA%\SnowRunnerNextGen**. Каждый новый файл записывается рядом со старым, читается обратно и проверяется, и только потом занимает место старого. Рядом с паками программа оставляет небольшую заметку, **SnowRunnerNextGen.json**, со списком файлов, в которых есть её изменения; кнопка Restore originals её удаляет.

Часть 6. Лицензия и авторы

Next Gen распространяется как свободное программное обеспечение по лицензии GNU General Public License версии 3 (GPL v3); её текст лежит в архиве, в файле LICENSE. Методы и код, на которых он построен, перечислены в **THIRD_PARTY_NOTICES.md** вместе с авторами и лицензиями, а на странице каждой карточки выше указано, чьи работы она использует. Файлы игры остаются собственностью Saber Interactive: Next Gen изменяет их на вашем ПК и не распространяет ни одного из них, кроме нескольких текстур, перечисленных в этом файле.